



Budapest University of Technology and Economics
Faculty of Electrical Engineering and Informatics
Department of Automation and Applied Informatics

Object Detection Techniques in Medical Imaging

BACHELOR'S THESIS

Author

Hadir Helali

Advisor

David Sik
Laszlo Grad Gyenge

June 1, 2025

Contents

Abstract

Résumé

1	Introduction	1
1.1	Overview	1
1.2	Background	1
1.3	Problem Statement	2
1.4	Motivation	2
1.5	Objectives	2
1.6	Thesis Structure	3
2	Literature review	4
2.1	Introduction to Object Detection and Machine Learning	4
2.2	Object Detection in Medical Imaging	5
2.3	Deep Learning and CNNs in Object Detection	6
2.4	One-Stage Detection: YOLO and Its Biomedical Applications	7
2.5	Two-Stage Detection: Faster R-CNN for Precision-Critical Tasks	9
2.6	Object Detection as an Alternative to Segmentation in Histopathology . . .	10
2.7	Placental Villi Analysis in Medical Imaging: The Research Gap	10
3	DataSet	11
3.1	OverView	11
3.1.1	Slide and Annotation Overview	11
3.2	Class Distribution and Imbalance Analysis	12
3.2.1	Class Overview	12
3.2.2	Observed Imbalance	12
3.2.3	Data Quality Notes and Methodological Impact	13
3.3	Annotation Conversion	13
3.3.1	Polygon Extraction	14

3.3.2	Bounding Box Conversion	14
3.3.3	Resulting Format	14
3.4	Tiling	14
3.4.1	Tiling for YOLOv8s	15
3.4.2	Tiling for Faster R-CNN	16
3.5	Augmentation and Oversampling	18
3.6	Data Split	18
4	Methodologies	20
4.1	YOLOv8s Model Architecture	20
4.2	YOLOv8s Training Mechanics and Strategy	21
4.2.1	Forward Pass and Predictions	21
4.2.2	Composite Loss Function	22
4.2.3	Training Schedule and Techniques	24
4.2.4	End-to-End YOLOv8s Pipeline and Technologies	25
4.3	Architecture and Model Configuration (Faster R-CNN)	26
4.3.1	Forward Pass and Output Heads	28
4.3.2	Loss Functions and Training Flow	29
4.3.3	Training Techniques and Augmentations	30
4.3.4	Inference Flow and Postprocessing	33
4.3.5	Pipeline Summary – Faster R-CNN	35
4.4	Postprocessing	36
4.4.1	Postprocessing Pipeline for YOLOv8s Inference	36
4.4.2	Postprocessing Pipeline for FasterRcnn Inference	37
5	Evaluation	42
5.1	Evaluation Metrics and Loss Functions	42
5.1.1	Confusion Matrix and Core Metrics	42
5.1.2	Intersection over Union (IoU)	42
5.1.3	Mean Average Precision (mAP)	43
5.1.4	Loss Functions in Object Detection Models	43
5.2	Yolov8s Evaluation	43
5.2.1	Sample Test Tile Visualizations	43
5.2.2	Inference and Test-Time Evaluation	45
5.2.3	Quantitative Results	46
5.2.4	Qualitative Results	47
5.2.4.1	Training and Validation Metrics Over Epochs	47

5.2.4.2	Confusion Matrix	48
5.2.4.3	Conclusion	48
5.3	Faster R-CNN Evaluation	49
5.3.1	Visual Prediction Examples on Test Tiles	49
5.3.2	Test Tile Inference and Visualization	50
5.3.3	Quantitative Results	51
5.3.4	Qualitative Results	52
5.3.4.1	Confusion Matrix	52
5.3.4.2	F1 vs Recall	53
5.3.4.3	Conclusion	54
5.4	Performance Comparison between YOLOv8s and Faster-Rcnn	54
6	Conclusion	56
	Acknowledgements	57
	Bibliography	58

STUDENT DECLARATION

I, *Hadir Helali*, the undersigned, hereby declare that the present **BSc** thesis work has been prepared by myself and without any unauthorized help or assistance. Only the specified sources (references, tools, etc.) were used. All parts taken from other sources word by word, or after rephrasing but with identical meaning, were unambiguously identified with explicit reference to the sources utilized.

I authorize the Faculty of Electrical Engineering and Informatics of the Budapest University of Technology and Economics to publish the principal data of the thesis work (author's name, title, abstracts in English and in a second language, year of preparation, supervisor's name, etc.) in a searchable, public, electronic and online database and to publish the full text of the thesis work on the internal network of the university (this may include access by authenticated outside users). I declare that the submitted hardcopy of the thesis work and its electronic version are identical.

Full text of thesis works classified upon the decision of the Dean will be published after a period of three years.

Budapest, June 1, 2025

Hadir Helali
student

Abstract

This thesis explores the use of object detection for automating placental microscopic analysis, focusing on the identification of Terminal, Intermediate, and Stem villi in high-resolution whole-slide images (WSIs).

Given the time-consuming nature and the inconsistencies of manual examination combined with the challenges imposed by the pixel-level annotation in segmentation techniques, villi classification is instead addressed as a bounding-box detection task.

The dataset preparation involved converting polygon-based annotations into bounding boxes, as well as breaking down the large image into small tiles, and applying augmentation and oversampling strategies to handle class imbalance.

Two object detection models YOLOv8 and Faster RCNN were trained and evaluated. YOLOv8 provided faster inference and strong performance on the most dominant classes, whereas Faster RCNN was better at reducing inter-class misclassification.

Despite challenges such as class imbalance and tissue complexity, results show that bounding-box detection is a viable and scalable alternative to segmentation in microscopic analysis.

Résumé

Cette thèse explore l'utilisation de la détection d'objets pour automatiser l'analyse microscopique placentaire, en se concentrant sur l'identification des villosités terminales, intermédiaires et souches dans des images de lames entières haute résolution (WSI).

Compte tenu du temps et des incohérences de l'examen manuel, combinés aux difficultés posées par l'annotation au niveau du pixel dans les techniques de segmentation, la classification des villosités est plutôt abordée comme une tâche de détection de cadres englobants.

La préparation du jeu de données a consisté à convertir les annotations polygonales en cadres englobants, à décomposer l'image en petites tuiles et à appliquer des stratégies d'augmentation et de suréchantillonnage pour gérer le déséquilibre des classes.

Deux modèles de détection d'objets, YOLOv8 et Faster RCNN, ont été entraînés et évalués.

YOLOv8 a fourni une inférence plus rapide et d'excellentes performances sur les classes les plus dominantes, tandis que Faster RCNN a permis de réduire plus efficacement les erreurs de classification interclasses. Malgré des défis tels que le déséquilibre des classes et la complexité des tissus, les résultats montrent que la détection par boîte englobante est une alternative viable et évolutive à la segmentation dans l'analyse microscopique.

Chapter 1

Introduction

1.1 Overview

The overlap between medical imaging and artificial intelligence has led to new strategies for automating complex diagnostic tasks in recent years. Object Detection, in particular, has become a go-to approach in identifying specific structures within medical images in a more efficient and faster manner, rather than relying on manual analysis.

Placental histology represents a promising yet underexplored area in medicine. Its proper classification can provide insights into the pregnancy, but current analysis methods remain non-time efficient and prone to variation between observers.

By integrating object detection, we can support the automated analysis of placental villi within high-resolution histological slides. By reframing villi identification as a bounding-box detection problem rather than a segmentation task, the goal is to see if bounding-box detection can reliably pick up what experts have already marked while still keeping the results meaningful from a biological point of view.

1.2 Background

The placenta is a short term yet essential organ that is developed during pregnancy to support fetal growth which enables the exchange of nutrients, gases, and waste between mother and fetus. When we examine it on a cellular level, it contains villous structures, which form an essential part of this exchange system.

These villi are classified into three categories; Terminal, Intermediate, and Stem villi, each one of them differ in morphology and function, their spatial distribution provides valuable diagnostic information about the health of the mother and fetus as well as the pregnancy progress.

Histopathological examination of placental tissue allows clinicians and researchers to examine these villous structures microscopically, often by using hematoxylin and eosin (HE) stained slides.

However, this process remains largely manual, requiring expert pathologists to visually inspect entire whole-slide images (WSIs), which are ultra-high resolution. They enable detailed exploration and annotation, but also represent some challenges due to their large size.

The digitization of histological slides through digital pathology and high-resolution scanners has opened new doors for computational analysis. Within this shift, object detection has become a practical alternative to traditional segmentation methods. Instead of needing pixel-level detail, object detection works with bounding boxes to find and classify structures, which makes things faster and more scalable, especially when expert annotations are already available.

1.3 Problem Statement

The detection process of the villi structure is not only resource-intensive but also subject to inconsistency between clinicians. While segmentation-based methods have been applied in medical imaging, their requirement for pixel-level annotations makes them impractical for this use case. Currently, there is a lack of streamlined, bounding box-based object detection approaches tailored to medical datasets that involve WSIs and have microscopic structures like placental villi, limiting progress in digital pathology workflows.

1.4 Motivation

Much of the research on object detection in medical imaging remains focused on a narrow set of clinical tasks with abundant, well-structured data. This leaves many important domains overlooked, particularly those involving less common structures, limited annotation resources, or high-resolution slides typical in digital pathology. There is a growing need to assess whether commonly used detection models can generalize well to such cases. This thesis is motivated by that question. Rather than tackling a widely studied application, it applies object detection to a more granular, high-resolution histological dataset, in our case, placental villi, and explores how well the model performs under real-world and medical conditions where annotations are hard to come by and structures are overlapping.

1.5 Objectives

This thesis aims to evaluate and compare the performance of two object detection models, YOLOv8 which is optimized for speed and Faster R-CNN, which is optimized for accuracy on a high-resolution medical imaging dataset. Although the dataset used concerns placental villi, the focus of the work is on evaluating object detection techniques in complex medical imaging scenarios, rather than addressing a domain-specific clinical question.

The specific objectives are:

- To prepare and preprocess a whole-slide histological dataset for object detection, including:
 - Converting polygon-based annotations into bounding boxes,
 - Tiling high-resolution images into model-compatible patches,
 - Addressing class imbalance through augmentation and oversampling.
- To implement and train two object detection pipelines based on YOLOv8 and Faster R-CNN.

- To evaluate both models using standard object detection metrics such as mean Average Precision (mAP), Intersection over Union (IoU), precision, and recall.
- To compare model performance under conditions typical in medical imaging, such as limited annotations and dense object distributions and evaluate bounding-box detection as a practical alternative to pixel-level segmentation.

1.6 Thesis Structure

- **Chapter 1:** introduces the research context, background, problem, motivation, and objectives.
- **Chapter 2:** reviews the relevant literature on object detection in medical imaging and it goes in depth in alignment with the work.
- **Chapter 3:** analyzes the dataset and the preprocessing pipeline: annotation conversion, tiling, and augmentation strategies.
- **Chapter 4:** describes the training of YOLOv8 and Faster R-CNN.
- **Chapter 5:** analyzes the evaluation results.
- **Chapter 6:** conclusion of the findings and directions for future work.

Chapter 2

Literature review

2.1 Introduction to Object Detection and Machine Learning

Machine learning is a pillar in artificial intelligence, which allows computers to learn from data and make predictions and decisions without the need for explicit programming for each assignment [13, 26]. When it comes to medical imaging, ML algorithms can process and assess huge volumes of image data like X-rays, MRI, and CT scans to detect abnormalities, classify diseases, and assist clinicians in diagnosis and treatment planning [13, 26]. Unlike traditional image analysis methods, which often rely on manual review by clinicians, ML models can efficiently process images and identify the small details that may be overlooked by the human eye, hence improving the accuracy and efficiency of diagnosis [25, 26].

A specialized subfield of ML, Deep Learning uses multi-layered neural networks to computationally learn hierarchical representations of raw data [13, 8]. When it comes to Medical Imaging, it has revolutionized the field, especially through convolutional neural networks, by enabling algorithms to autonomously extract complex features from images without the need for manual feature engineering [13, 25, 8]. These Networks excel at capturing complex spatial patterns in medical images, making them exceptionally effective for tasks involving segmentation, classification, and most importantly, object detection. Deep Learning Models trained on large and annotated datasets have demonstrated high levels of accuracy in identifying and classifying anomalies across various medical imaging tasks, this helps to deliver more precise and faster interpretations that support better patient care [8, 25].

One of the many applications of deep learning in medical imaging is object detection. It involves the localization and classification of multiple objects or regions of interest within a single image [26]. As opposed to image classification, that mainly assigns a single label to an entire image, object detection aims to identify the category and the position of each significant object such as tumors, lesions or anatomical landmarks by assigning class labels and drawing a bounding box around them [6].

These twofold functionalities are trivial for multiple clinical tasks, from lesion localization in endoscopy to cancer detection in pathology slides. The integration of ML, deep learning and bringing object detection into medical workflows has elevated the field to new highs and enabled earlier detection and more tailored treatment, and it has also improved patient care through automated, accurate and scalable image analysis [25, 26, 8].

2.2 Object Detection in Medical Imaging

The progress of object detection in medical imaging has gone hand in hand with improvements in deep learning especially convolutional neural networks (CNNs). These CNN-based methods are now a core part of modern medical image analysis, helping automatically detect important disease-related features with high accuracy and speed. The process usually has two steps: first, the model will extract the features from the image, then it will classify and locate the relevant objects. CNNs are great for this because mainly they can learn simple and complex visual patterns that are vital for medical diagnosis [6].

Object detection is generally categorized into two main frameworks: two-stage and onestage detectors. Two-stage detectors, like Faster R-CNN, begin by generating region proposals, which are then classified and refined. This method is well known for its high accuracy and is particularly effective in cases where precise localization is vital such as identifying small or overlapping lesions in CT or MRI scans [6, 27]. Faster RCNN has been successfully applied to detecting metastatic lymph nodes in CT images, showing strong results in both recall and mean average precision (mAP) [6, 28, 27]. The architecture includes a CNN backbone for feature extraction, a Region Proposal Network (RPN) to generate candidate regions, and a classification head to assign labels and finetune bounding boxes all integrated into a single end-to-end trainable system[28, 27].

One stage detectors, such as the YOLO (You Only Look Once) family, the object detection pipeline by handling localization and classification in a single pass. This results in significantly faster inference, which makes YOLO-based models especially attractive for real-time applications and large-scale screening scenarios[6, 9, 29, 10]. Latest versions, such as YOLOv8 and MedYOLO, have further enhanced the detection performance and efficiency in medical contexts, including 3D imaging tasks [9, 14, 10]. The YOLO models have been used to detect a variety of targets like brain tumors in MRI scans and blood cells in microscopy images and have demonstrated high sensitivity and consistent performance in multiple clinical settings [29, 15, 16, 10].

Choosing between these two frameworks usually comes down to the designated needs of the clinical task. Two-stage models like Faster R-CNN are often the most reliable when our focus is accuracy. On the other hand, one-stage models like YOLO are better suited for situations where speed and scalability are paramount. That said, both types still face common challenges like the reliance on large and expertly annotated datasets and the difficulty of picking up very small or rare structures[9, 14, 15]. In general, object detection algorithms, especially CNN based ones like Faster RCNN and YOLO have become a core part of medical image analysis; they allow for automated, accurate, and scalable detection of critical features, helping support earlier diagnosis and more tailored treatment. Research in this space is still moving fast with ongoing efforts to improve data annotation, model interpretability, and performance on subtle or rare findings, paving the way for even better integration of AI into clinical workflows[6, 9, 14, 27, 15, 10].

2.3 Deep Learning and CNNs in Object Detection

The Role of CNNs in Object Detection CNNs are built in layers that are essential to capture both spatial and contextual details in images, which makes them effective when working with complex medical data. In object detection, they’re the part that does the heavy lifting, by taking the raw image and turning it into rich feature maps that capture everything: simple edges and textures as well as complex shapes and patterns. These features are then used to figure out what’s in the image and where it is, for example, identifying a tumor or a specific anatomical structure [6, 17].

The backbone architecture corresponds to the main CNN used for feature extraction in object detection models. Some architectures have become conventional in the field because they offer a good trade-off when it comes to accuracy, speed, and how well they generalize across different types of medical imaging tasks

- **VGG16/VGG19** are among the earliest but still widely used backbones. Mostly well known for their simplicity, as they use small convolutional filters with a consistent structure throughout the network. Even though their efficiency does not match today’s standards, they’ve been used in models like Fast R-CNN and SSD and still serve as a solid baseline for many medical imaging tasks [18].
- **ResNet:** Introduced residual connections, which help the model go deeper without running into the vanishing gradient problem. Variants like ResNet-50 and ResNet101 are very common in frameworks like Faster R-CNN and Feature Pyramid Networks (FPN). These backbones perform well in tasks that specifically involve detecting small or subtle features, like microcalcifications in mammography [17, 18].
- **DenseNet:** Connects each layer to every other layer in a feed-forward way, which helps to reuse features and to improve gradient flow. It has shown impressive outputs in areas like histopathology and volumetric imaging, where picking up very fine details is important [17].
- **Inception, Inception-ResNet, and EfficientNet:** Take things further by combining multi-scale feature extraction with adjustments in depth and width. Among them, EfficientNet stands out for reaching high accuracy while keeping the model lightweight, which is especially useful in clinical settings where resources are limited, or real-time performance is needed [21].

The choice of backbone usually depends on multiple factors: the specific clinical use case, the type of imaging modality, as well as the available computational resources. For tasks that demand high sensitivity, deeper models are usually preferred. But in scenarios that require quick decisions like point-of-care tools, lighter and faster networks tend to be a better fit [21, 11].

Training Strategies in Medical Contexts Training CNNs for medical object detection comes with a few well-known challenges, like limited annotated data, class imbalance, and the need for models to be consistently reliable in clinical environments. To work around these, the field uses several well-established strategies.

- **Transfer Learning:** is one of the most commonly used approaches. Instead of starting from scratch, Leveraging models pre-trained on large datasets like ImageNet and fine-tunes it on medical images. This speeds up training and helps the model generalize better especially when labeled medical data is hard to get. [17, 19].
- **Fine Tuning:** helps adapt pre-trained models to specific medical tasks. it only retains the layers that matter for the task and freezes the rest, the model is able to pick up relevant patterns without the need to overfit to the limited dataset. [18, 17].
- **Data Augmentation:** is a simple but efficient way to deal with limited data. Using techniques like rotating, flipping, scaling or changing brightness helps to create and generate more training data , which enables the model become more robust with low sensitivity to small variations [19].
- **Handling Class Imbalance:** is also hugely important, especially when rare cases (like abnormalities) count only a little percentage of the data. Methods like using weighted loss functions, oversampling, or generating synthetic data is helpful for the model to avoid bias toward the majority class [19].
- **Semi-Supervised and Federated Learning:**are also gaining traction. These approaches let us make use of unlabeled data or train models across decentralized datasets, something that's especially useful in medical settings where data sharing is often restricted.[19].

Clinical Impact and Future Outlook CNN-based object detection has brought real improvements to medical imaging by helping doctors catch diseases earlier, plan treatments more precisely, and support better outcomes for patients across CT, MRI, X-ray, and microscopic analysis. Still, there are some obstacles to overcome like the heavy need for annotated data, high computing power, and the challenge of understanding how these models make decisions. Looking ahead, explainable AI and combining data from different sources seem like exciting paths forward, both for earning clinical trust and for making these tools more useful in real world healthcare settings. [17, 20, 6].

2.4 One-Stage Detection: YOLO and Its Biomedical Applications

The You Only Look Once (YOLO) family is widely popular due to its simple and efficient one stage detector design. Unlike two stage models like Faster R-CNN which first generate region proposals and then classify them YOLO handles everything in one go. It predicts both the location and class of objects in a single forward pass. This makes it incredibly fast while still maintaining solid accuracy, which is exactly what's needed in fields like biomedical imaging where both speed and scale matter [22].

YOLOv5 was a big leap forward in terms of ease of use and flexibility. It quickly became the go to version for many researchers and developers, largely because of its clean, modular PyTorch codebase. In the medical world, it's been applied successfully to a wide range of tasks like spotting malaria parasites in blood samples[3], identifying skin lesions [12], and detecting signs of COVID-19 in chest X-rays [5]. These use cases show just how well YOLOv5 is adaptive to multiple biomedical imaging problems, even if It's working with little training data.

One of the latest official releases, YOLOv8, brings substantial improvements, making it especially suitable for complex biomedical applications like histopathology. YOLOv8 brings in an anchor free design with a decoupled head for detection and improved loss functions for bounding box regression and classification. This helps it to do a better job at locating small and irregular structures, crucial for detecting placental villi. Additionally, YOLOv8 supports instance segmentation and classification as optional tasks, which helps to expand its flexibility in medical pipelines. As well, it helps to automatically resize the input and to export the model efficiently for deployment [22].

When domain-specific adaptations like MedYOLO have been introduced, they integrated specific priors and adaptations tailored for medical datasets [7]. However, even general use of YOLOv8 has shown strong performance in detecting tumors, cells, and lesions, as it can efficiently handle a variety of object sizes and dense regions [22].

At a high level, YOLO’s architecture is made up of three parts: a backbone, a neck, and a head. The backbone (CSPDarknet or similar) its role is to extract the features from the input image/data. The neck (such as PANet or FPN) aggregates and passes multi-scale features onward to facilitate the detection of objects with diverse spatial scales. Finally, the head performs the actual detection by mainly predicting bounding boxes, objectness scores, and class probabilities. YOLOv8 simplifies this even further by using an anchor free head and modern convolutional layers, resulting in faster inference and cleaner training dynamics [22].

How Anchor-Free Detection Works Anchor-Free models such as FCOS (Fully Convolutional One-Stage Object Detection) follow this process [4]:

- **Feature Extraction:** aims to extract the feature map which is what the model sees as layers; from an input image.
- **Center Point Prediction:** For each location in the feature map (layers), the model’s aim is to assess whether it is at the center of the object or not.
- **Bounding Box Regression:** If a center point is detected, the model directly predicts the distances from this point to the four sides of the bounding box (top, bottom, left, right)

2.5 Two-Stage Detection: Faster R-CNN for Precision-Critical Tasks

Faster R-CNN is a two-stage object detection framework acknowledged for its high accuracy when it comes to tasks requiring precise localization of small or overlapping objects [4]. Its architecture is built on three components:

- **Convolutional Backbone:** An initial convolutional neural network (ResNet, VGG) processes the entire input image to extract a comprehensive feature map [4].
- **Region Proposal Network (RPN):** It scans the feature map to propose regions of interest (ROIs) that will eventually enclose the objects. It outputs a set of bounding boxes with predicted object confidence values which effectively narrows down areas for detailed analysis [4].
- **ROI Pooling and Classification Head:** after the model picks a region of interest (ROI), ROI pooling kicks in to turn that region's features into a fixed-size vector. This makes it easier to handle. That vector is then passed through fully connected layers, which are responsible for classification to identify the object and refine the bounding box coordinates [4].

The two-stage nature of Faster R-CNN allows for careful examination of each proposed region, making it exceptionally capable at detecting objects that are small, densely packed or overlapping common characteristics of placental villi in histological images. In biomedical imaging, Faster R-CNN has demonstrated superior performance in various applications for example:

- **Lesion Detection:** It helps to detect lesions in liver CT scans and brain MRIs, which contributes to early diagnosis [2].
- **Cellular Structure Recognition:** involves identifying and classifying various cellular components in histopathological images [23].

Compared to one stage detectors like YOLO, Faster RCNN delivers higher localization accuracy, but with the downside of increased computing power and inference time. Still, that accuracy is the most valuable tool in medical tasks where precise detection is paramount [24].

2.6 Object Detection as an Alternative to Segmentation in Histopathology

Image segmentation, in histopathology, has been a key technique in analyzing microscopic tissue structures. Segmentation models traditionally aim to assign a label to each pixel in an image, facilitating highly detailed delineation of objects like nuclei. Techniques such as U-Net or Mask R-CNN have been widely used to isolate cellular structures with impressive pixel-level precision, making it possible to measure the size, shape and position of structures; details that are really important for some diagnostic processes [1].

Despite that, segmentation comes with its own practical challenges, more specifically in large-scale applications. Pixel-wise annotation is not time efficient and requires expert input, which makes the scalability of training data limited. Moreover, segmentation models are often computationally intensive and require careful post-processing to extract usable object-level information from the raw pixel masks. In use cases where the end goal is to only detect, classify, or count distinct structures (rather than outlining them) its complexity may not be necessary and could be avoided [24].

This is where object detection offers a compelling alternative. Instead of aiming to label each pixel, the detection models like Faster R-CNN or YOLO assign bounding boxes around objects of interest [4]. These bounding boxes provide sufficient localization for many analysis goals, while significantly limiting the burden of annotation. In the context of placental histology, villi structures can be meaningfully located and classified using bounding boxes without needing exact outlines especially when the biological endpoint is to estimate count or categorize type rather than analyze its shape [23].

For this reason, object detection is a practical and scalable approach for villi classification. The bounding-box-based method aligns well with the existent annotations, which were given as polygons and later on converted to detection-friendly format.

2.7 Placental Villi Analysis in Medical Imaging: The Research Gap

Even though placental villi hold biological significance in monitoring pregnancy health and development, they have not received much attention in computational medical imaging research. Most histopathology studies focus on cancer diagnosis, gland segmentation, or cellular structure detection, while villi-specific analysis using automated object detection has been overlooked [23, 1].

When villi are studied on a computational level the most often used strategies are segmentation methods; used to isolate regions of interest, but they lack scalability as we mentioned earlier since they require pixel level annotation [1]. Meanwhile, In clinical and pathological settings, identifying villi types depends on visual assessment around their shape, size and their context under a microscope which is an approach that can be easily mimicked by deep learning models trained on bounding boxes and visual features [4, 23].

This thesis addresses the research gap by introducing an object detection pipeline designed specifically to locate and classify placental villi in histological slides. By converting polygon annotations to bounding boxes and leveraging models like YOLOv8 and Faster RCNN, the system mimics expert classification in a more adaptable and reproducible way [24].

Chapter 3

DataSet

3.1 OverView

The dataset used in this study originates from Semmelweis University and comprises histological slides of human placenta tissue, to be more precise, from a pregnancy around the 27th week. These samples were stained using Hematoxylin and Eosin (H&E), which is a standard technique in pathology to highlight cell nuclei in blue and other tissue in different shades of pink. This staining provides clearer visual contrast and it enables the differentiation of components as the villi. The images were stored in a digital pathology archive accessible through a secure server connection to the hospital's internal system. Access and download are managed via a software called SlideMaster, which also facilitates image retrieval and its metadata extraction. The slides themselves are viewed using a complementary tool, SlideViewer, designed specifically for high-resolution microscopy data.

Each slide was made available in three formats:

- **MRXS:** A proprietary format used by 3DHISTECH scanners, optimized for multi-resolution viewing which allows smooth zooming and panning without the need to store its full resolution all at once.
- **TIFF:** a standard and widely supported format to store whole slide images (WSI). It preserves full image quality without compression, which is suitable for use in computer vision pipelines.
- **XML:** a structured annotation file containing spatial coordinates of manually labeled villi, including their polygon boundaries and their associated metadata.

The XML annotations have a crucial role in this work, as they contain expert-curated markings of villi that serve a vital input for any object detection model seeking to learn the morphology and spatial patterns.

3.1.1 Slide and Annotation Overview

All data provided in this study originates from a single patient and it represents the same placental tissue, which ensured consistency across slides. Within the SlideViewer software, the whole slide image can be seen organized into six large rectangular regions that we refer to as annotations ; which will be global zones of interest.

However, only Annotation1 and Annotation2 were extensively labeled at the villus level. As a result, these two regions form the base dataset. The slides are high-resolution whole slide images (WSI) captured via high-magnification microscopy: Annotation1 is at $8192 \times 15,104$ pixels, and Annotation2 is at $6912 \times 13,568$ pixels. This high level of detail is important for detecting the characteristic morphological structures.

The Hematoxylin and Eosin (H&E) staining is consistent across both slides, producing high contrast between nuclei and cytoplasmic tissue. The staining quality is visibly uniform, with minimal artifacts which still makes the slides making the slides appropriate for automated image analysis.

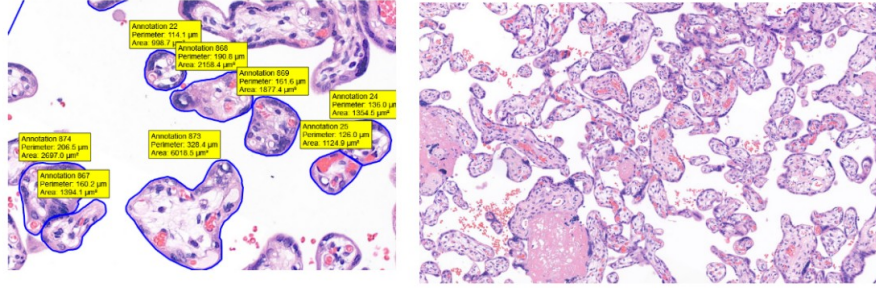


Figure 3.1: Annotations on SlideMaster.

Annotation details are provided in XML format, where each villus is marked by a polygon defined through its exact coordinate points. These polygon masks are yet to be converted to bounding boxes; they contain the full metadata required for region detection. Alongside each XML file, two separate Excel spreadsheets were provided to map each annotation ID to its corresponding villus type (Terminal, Intermediate, or Stem) and to its source region (Annotation1 or Annotation2). These files had to be proofread and combined into one single file, during the process some inconsistencies and duplicate IDs were discovered and corrected.

The annotation density, however, varies between the two slides: Annotation1 contains a higher number of villi, while Annotation2 includes more instances of the rarer and less existent villus types.

3.2 Class Distribution and Imbalance Analysis

3.2.1 Class Overview

The annotated dataset used in this study includes three biologically distinct types of placental villi: Terminal, Intermediate, and Stem villi. Each instance was labeled individually based on the polygon annotations that has been done by pathologists. The mapping of annotation IDs to villus type and source region (Annotation1 or Annotation2) was provided through two accompanying Excel sheets; which I had preprocessed and merged into one file to facilitate the label mapping during model training.

3.2.2 Observed Imbalance

The dataset shows a clear class imbalance. Terminal villi is the most dominating, accounting for over 70% of all labeled objects. Intermediate villi appear less frequently

but still have a decent presence in comparison to the Stem villi which are exceedingly rare, with only 16 instances across both slides. This imbalance is clearly evident and presents significant challenges in training models , which tend to learn dominant class patterns and perform poorly on the scarce ones.

This imbalance also differs between slides with Annotation1 is denser and Terminal villus heavy, while Annotation2 covers more villus types but fewer total samples.



Figure 3.2: Statistics extracted from the combined excel sheet of Annotation1 and Annotation2.

3.2.3 Data Quality Notes and Methodological Impact

Preprocessing revealed 13 duplicate annotation IDs across the two Excel files ; these were removed to ensure uniqueness .On top of that, one annotation was assigned to two different villus types, which was considered invalid and excluded from the dataset due to its label inconsistency . These adjustments were necessary to maintain clean data and most importantly to avoid conflicting inputs during training.

The class disparity , especially with Stem villi, significantly affected the model behaviour. Early experiments showed that the model had high confidence and precision for Terminal villi, while Intermediate and Stem villi were either missed or mislabeled . Object detection models like YOLOv8 rely on class frequency meaning the dominant class gradients shape the training process which comes at the expense of minority classes and can lead to failure to learn representative features.

3.3 Annotation Conversion

The original villi annotations were provided in XML format, where each object was marked as a polygon consisting of (x, y) coordinate points outlining each individual villus within the tissue slides. Since object detection models work with bounding boxes , the polygons had to be converted into axis-aligned rectangles , along with corresponding class labels. This process was done separately for the YOLOv8 and Faster R-CNN models, but both followed the same core logic.

The merged Excel sheet (Annotation1And2Combined.xlsx) was used to map each annotation_id to its villus class and its source slide (Annotation1 or Annotation2). The class labels were: Terminal villi → 1, Intermediate villi → 2, Stem villi → 3

3.3.1 Polygon Extraction

Using Python's `xml.etree.ElementTree` library, the XML files for both Annotation1 and Annotation2 were parsed. Each `<annotation>` entry contained a unique name (ex: "Annotation 10") and a list of `<p>` points which defined the polygon shape. In the YOLOv8s pipeline, the polygons were extracted and matched with their string based class label (ex: "Terminal villi") from the Excel sheet. In the Faster R-CNN pipeline, the polygons were instead linked to a numerical label (1, 2, or 3), compatible with PyTorch's detection framework.

3.3.2 Bounding Box Conversion

The polygons were converted into bounding boxes by taking the minimum and maximum x and y coordinates. The output format was:

$$[x_{\min}, y_{\min}, x_{\max}, y_{\max}]$$

for both Faster RCNN and YOLOv8 (later normalized in the YOLO pipeline). During extra validation steps, empty or broken annotations were removed. Any `annotation_id` not found in the Excel sheet or that had conflicting labels was excluded.

3.3.3 Resulting Format

For YOLOv8, the output was saved as a list of dictionaries containing the class names and bounding boxes, which were later tiled and converted to YOLO-format (.txt files) where each annotation is written as:

$$\text{class_id } x_{\text{center}} \ y_{\text{center}} \ \text{width} \ \text{height}.$$

For Faster R-CNN, each dictionary contained the .tif image name, numeric label, and bounding box. These were later grouped by image as Pytorch's FasterR-CNN model expects all annotations of one image to be passed on together during training. The format differences were needed because each model handles its input differently :

Yolov8 supports flexible preprocessing , which defers label assignment to the export phase while Faster R-CNN requires organized input with numeric class labels and grouped annotations to comply with its data loader requirements .

This conversion step ensured that both models had consistent and valid inputs derived from the original , and uniformly labeled training data derived from the labeled polygon inputs.

3.4 Tiling

Starting off , tiling refers to the process of dividing high-resolution images into smaller overlapping patches (tiles) while maintaining spatial coherence across boundaries. Unlike simple non-overlapping slicing, tiling ensures that objects near tile borders are not lost from the dataset. This is particularly important in histological data and medical data in general , where key structures like villi may show up right along the tile boundaries

Since the original slides are super high-res (8192×15104 for Annotation1 and 6912×13568 for Annotation2), it was not practical to use them directly for training. Two distinct tiling strategies were developed for the YOLOv8s and Faster R-CNN pipelines in accordance with their architecture .

3.4.1 Tiling for YOLOv8s

In the YOLOv8s pipeline, the slides were tiled into 1024×1024 patches with a 25% overlap (stride = 768). This size was a good middle ground to ensure that smaller villi were not missed at tile borders, that the Object localization remained precise and more importantly the number of generated tiles would not overload the system.

```
tile_size = 1024
overlap = 0.25
stride = int(tile_size * (1 - overlap)) # 768

for top in range(0, height, stride):
    if top + tile_size > height:
        top = height - tile_size
    for left in range(0, width, stride):
        if left + tile_size > width:
            left = width - tile_size

        tile = img.crop((left, top, left + tile_size, top + tile_size))
        tile_id = f"{source}_tile_{tile_counter:05d}"
        tile.save(...)
        tile_info[tile_id] = {
            "source": source,
            "x_offset": left,
            "y_offset": top,
            ...
        }
```

Listing 3.1: Tiling logic with 25% overlap and final offset handling.

Bounding Box Assignment Each original bounding box was checked for intersection with every tile from its source image. If it did then we would recenter and shift the box to match the tile-local space, as well as clip its parts that fell partially outside so it actually fit inside ; the boxes were then saved in YOLOv8 format (class_id x_center y_center width height).

```

for ann in all_annotations:
    global_bbox = ann["bbox"]

    for tile_id, tile in tile_info.items():
        if ann["source"] != tile["source"]:
            continue # skip other source

        tile_box = (
            tile["x_offset"],
            tile["y_offset"],
            tile["x_offset"] + tile["width"],
            tile["y_offset"] + tile["height"]
        )

        if boxes_intersect(global_bbox, tile_box):
            adjusted_bbox = adjust_bbox_to_tile(global_bbox, (tile["x_offset"],
tile["y_offset"]))
            tile_annotations[tile_id].append({
                "class": ann["class"],
                "bbox": adjusted_bbox})

```

Listing 3.2: Mapping global annotations to their corresponding tiles.

In this snippet, you can see that we iterated through each bounding box from the original annotation, and we checked for intersection with each tile that belongs to the same source image. The boxes that intersected got recentered according to the tile's local coordinates and adjusted to fit into the tile bounds.

Tiles with no bounding boxes were dropped to eliminate noise and avoid training on empty regions; only tiles containing at least one object were kept.

Tile images were saved in PNG format to preserve image quality and maintain compatibility with subsequent stages. JPEG was intentionally not used due to its lossy compression, which can lead to losing important details.

3.4.2 Tiling for Faster R-CNN

```

for top in range(0, img_h, stride):
    for left in range(0, img_w, stride):
        right = min(left + tile_size, img_w)
        bottom = min(top + tile_size, img_h)

        tile_boxes = []
        for b in relevant_boxes:
            x_min, y_min, x_max, y_max = b['bbox']
            if x_max < left or x_min > right or y_max < top or y_min > bottom:
                continue

            adj_xmin = max(x_min - left, 0)
            adj_ymin = max(y_min - top, 0)
            adj_xmax = min(x_max - left, tile_size)
            adj_ymax = min(y_max - top, tile_size)

            if adj_xmax <= adj_xmin or adj_ymax <= adj_ymin:
                continue

```

```

tile_boxes.append({...})

if len(tile_boxes) >= min_objects_per_tile:
    tile_img = img.crop((left, top, right, bottom))
    tile_img.save(...)
    tile_annotations.append({...})

```

Listing 3.3: Sliding window tiling with bounding box adjustment.

At first the Faster R-CNN pipeline followed the same 1024×1024 tiling approach , similar to YOLOv8s. However, this setup revealed several limitations : Large or irregular villi, such as stem villi, were often fragmented across adjacent tiles , Important surrounding tissue context was lost, which negatively affected the region proposal network’s ability to locate and identify full structures.

To avoid these issues, we went up to 2048×2048 tiles with 50% overlap (stride = 1024). This larger tile size leveraged the model’s multi-scale feature extraction, and helped to reduce fragmentation and capture richer context .

Bounding Box Adjustment Bounding boxes overlapping a tile were adjusted (shifted and clipped as we have seen done previously in yolov8s) or discarded if they were invalid. Only tiles containing at least one valid bounding box were kept, and the tiles were saved in .png; All for the same reasons as mentioned in yolov8s.

Although both pipelines operated on the same data, their unique model architectures required different preprocessing strategies. YOLOv8s, being a fast one-stage detector , benefited from smaller tiles (1024×1024) that isolated local features and reduced GPU load.

In contrast, Faster R-CNN, a two-stage detector, was better equipped to process larger tiles (2048×2048), using its region proposal network (RPN) to capture broader context. Each setup was tailored with each model that played to its strengths .

Resulting Annotation Format For YOLOv8s, each tile had its own .txt file containing the normalized bounding box and its corresponding class ID. For Faster R-CNN, a single annotations.json was used, containing a list of tile records with image paths, bounding boxes, numeric labels, and source information.

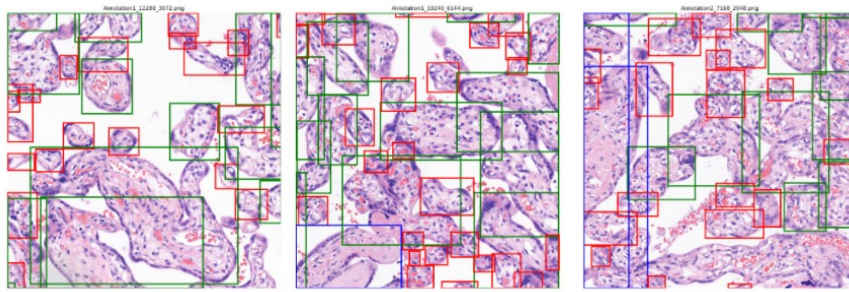


Figure 3.3: Merged Grid of 2048×2048 Tiles with Annotations .

3.5 Augmentation and Oversampling

This section will focus on the steps taken to decrease the class imbalance and reduce bias in the training through a variety of steps ; including firstly augmentation which will create different versions of the same data by applying tweaks like flips and color adjustments ; oversampling which will give extra weight to underrepresented classes by duplication → combining these methods together helps the model avoid overfitting and just memorizing the training data.

After tiling, the next critical step was to address the dataset’s class imbalance and limited size through augmentation and oversampling . Given that terminal villi are overrepresented in comparison to Intermediate and Stem villi, both pipelines used methods customized to their respective architectures and data formats.

For YOLOv8s, augmentation was handled directly by the Ultralytics training engine. Each training tile underwent basic transformations such as horizontal and vertical flips, slight rotations, translations, and shearing. Add to that, techniques like Mosaic and MixUp were used to blend multiple tiles, simulating complex visual arrangements that could enhance the model’s resilience to spatial irregularities . These augmentations were exclusively applied to the training set to ensure an unbiased evaluation on validation and test data .

To further minimize the impact of class imbalance, we used an oversampling strategy. Tiles that contained at least one Stem villus were duplicated three times in the training set. This duplication helped to fight the imbalanced distribution and allowed the model to train better from underrepresented classes while maintaining the original labels and bounding boxes.

For Faster R-CNN, data augmentation was handled explicitly through PyTorch’s torchvision.transforms. The training pipeline implemented augmentations such as random horizontal and vertical flips and color jittering ; without compromising the biological integrity while at the same still introducing enough variation to reduce overfitting. As with YOLOv8s, augmentation was only applied to the training set to preserve the validity of evaluation results.

When it came to oversampling , tiles with Intermediate or Stem villi were duplicated four times during training data preparation. This strategy boosted the minority classes, by improving their visibility during training as well as aiming to achieve class uniformity in the learning process.

Since each model has its own input style , the annotations were formatted differently : YOLOv8s saved per-tile .txt files with normalized bounding box coordinates and class IDs , while Faster R-CNN relied on an annotations.json file that grouped raw bounding boxes, class labels, and metadata for each tile.

Together, these steps made up the final stage of data preprocessing . The end goal here was to increase the diversity, balance, and effectiveness of the training data, with tweaks made to fit how each model worked best , taking advantage of its architectural strengths to the maximum .

3.6 Data Split

In supervised learning , we teach the model on the training set, and we use the validation set to tune hyperparameters and check for overfitting during training, and the test set provides an unbiased estimate of the model’s performance . While YOLOv8s used an

80/10/10 split and Faster R-CNN used 70/15/15, the difference was based on practical considerations.

YOLOv8s, being a single-stage detector, benefits from maximizing training data to recognize patterns. In comparison, Faster R-CNN, due to its multi-stage architecture and longer training process, benefits from a larger validation set to support stable performance and easier tuning. Both pipelines started by randomly shuffling the tiled images before splitting them.

The strategy of shuffling the tiles guarantees that the training, validation, and test sets are randomly distributed, which prevents any bias that could arise if, for example, neighboring tiles from the same slide ended up in the same set, which could lead to overfitting or unrealistic evaluation.

Model	Split	Before Oversampling	After Oversampling	Format
YOLOv8s	Train	305	725	.txt, .yaml
	Validation	38	38	
	Test	39	39	
Faster R-CNN	Train	218	608	.json
	Validation	33	33	
	Test	33	33	

Table 3.1: Tile counts before and after oversampling for YOLOv8s and Faster R-CNN datasets.

In the YOLOv8s pipeline, a fixed random seed was used so that the split would be consistent across runs. Image-label pairs were stored in YOLO's format with annotations in .txt format and a corresponding data.yaml configuration file. Only the training set was oversampled, the tiles that included Intermediate or Stem villi were replicated four times to decrease class imbalance. However, the validation and test sets remained unchanged to preserve evaluation integrity.

Chapter 4

Methodologies

4.1 YOLOv8s Model Architecture

As we outlined in Section 2.4, YOLOv8 introduces a simpler, anchor-free architecture that is well-suited for object detection tasks in biomedical imaging. YOLOv8s variant, a lightweight version of the model was chosen because of its efficiency and compatibility with high-resolution image tiles (1024×1024), which made it suitable to accurately detect the placenta villi without having to consume too much computational power.

The YOLOv8s architecture is built around three main components: a C2f-based backbone, a feature aggregation neck, and an anchor-free detection head for final predictions (see Figure 4.1). We started with the official `yolov8s.yaml` configuration and trained from scratch to ensure that it could learn patterns specific to villi detection.

Backbone C2f Modules: YOLOv8 introduces C2f (Cross-Stage Partial Fusion) as a replacement for the CSPDarknet backbone implemented in the earlier YOLO versions. These modules help improve gradient flow, cut down on unnecessary computations, and promote more effective learning by blending shallow and deep representations [22]. This structure is particularly helpful in medical images where the objects tend to be small and irregularly shaped.

Neck PANet-style Feature Aggregation: The neck uses a PANet-style feature pyramid [4] to merge feature maps from different layers of the backbone. This improves the detection of villi at different spatial scales, which is important when we are dealing with diverse morphological states in biomedical images.

Head Anchor-Free, Decoupled Design: The anchor-free head predicts bounding boxes directly from feature maps without depending on predefined anchor templates. Each location in the feature map is treated as a potential object center, and bounding box offsets are predicted directly by regression branches, following techniques introduced in FCOS [4]. This is particularly beneficial in biomedical domains where object sizes and aspect ratios vary highly. Furthermore, YOLOv8 separates the classification and localization into distinct branches, improving learning stability and minimizing mutual interference [22].

As an input-size agnostic architecture, YOLOv8s was configured to process 1024×1024 tiles, to preserve spatial context details necessary for biomedical analysis. Its lightweight

structure ensures faster inference, making it practical for deployment in clinical research pipelines.

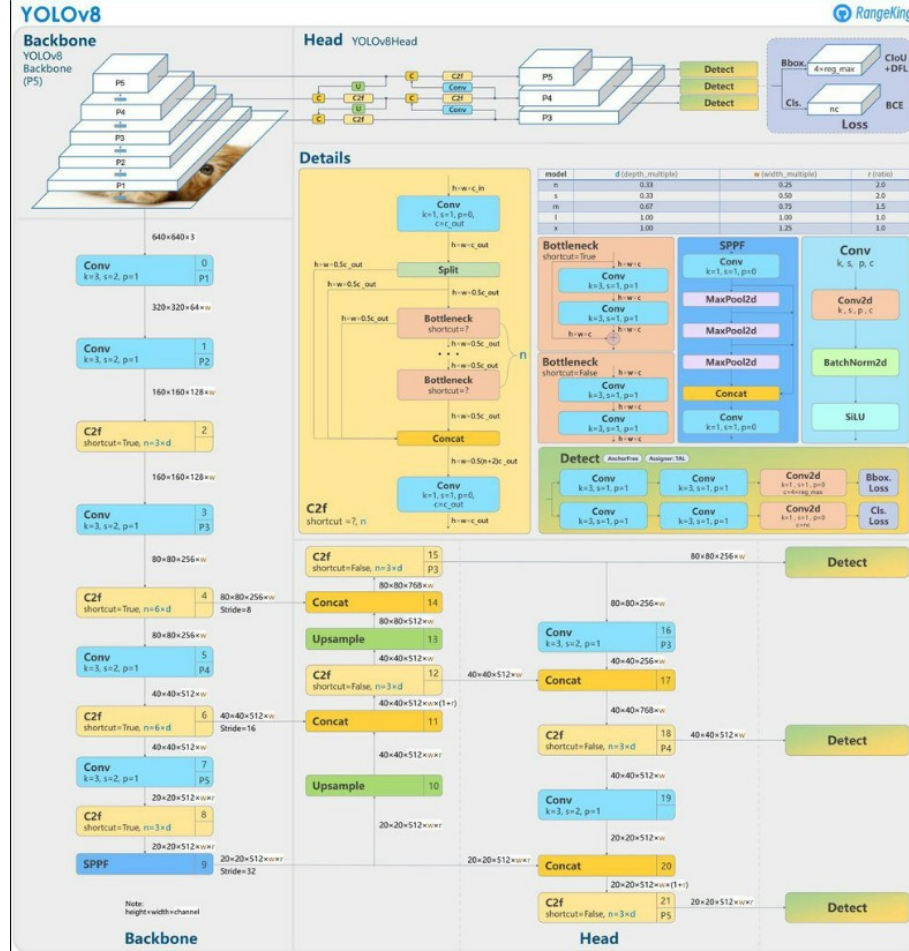


Figure 4.1: YOLOv8 Architecture. [30]

4.2 YOLOv8s Training Mechanics and Strategy

The training process for YOLOv8s model focuses on gradually and iteratively optimizing as well as adjusting its weights to improve bounding box placement and class identification. In this section, we will explain the core components of the training flow, data augmentation techniques, key loss functions and the optimizer settings as originally implemented in the Ultralytics framework of YOLOv8.

4.2.1 Forward Pass and Predictions

Each input tile (1024×1024) is passed through the model's C2f-based backbone, which learns layered features from the image. These are then merged by the PANet-style neck to create multi-scale representations. Finally, the anchor-free, decoupled head processes this information and produces three parallel outputs:

Bounding box regression: For each detected object the model outputs the normalized box coordinates (x, y, w, h) , where

$$x, y \rightarrow \text{center coordinates}, \quad w, h \rightarrow \text{width and height},$$

and all values are relative to the input image dimensions.

YOLOv8 uses the Complete Intersection over Union (CIoU) loss during training to refine the bounding box predictions. CIoU goes beyond simple IoU by also considering IoU overlap between predicted and ground truth boxes, the distance between box center points, and their aspect ratio differences. The loss function is defined as

$$\mathcal{L}_{\text{CIoU}} = 1 - \text{IoU} + \frac{\rho^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2} + \alpha v,$$

where $\mathbf{b}, \mathbf{b}^{gt}$ represent predicted and ground truth boxes, ρ is the Euclidean distance between box centers, c is the diagonal length of the smallest enclosing box, v is the aspect ratio consistency term, and α is a trade-off parameter.[31]

Objectness score: At each spatial location, YOLOv8 predicts the presence of an object regardless of class via

$$P_{\text{obj}} = \sigma(z_{\text{obj}}),$$

where z_{obj} is the raw output logit and σ is the sigmoid activation. Unlike earlier YOLO versions, this score is separate from IoU or class probabilities, acting as an independent binary classifier to improve stability.[30]

Class probabilities: The model also predicts class scores independently using sigmoid activation, supporting multi-label classification:

$$P_{\text{cls},i} = \sigma(z_{\text{cls},i}),$$

where $z_{\text{cls},i}$ is the raw logit for class i . This design allows YOLOv8 to accommodate overlapping or ambiguous categories, useful in histopathological analysis where features may be shared across villi types. [30]

YOLOv8’s design separates objectness, localization, and classification tasks, avoiding hand-tuned anchors and providing clearer gradients compared to earlier versions.

4.2.2 Composite Loss Function

Training the YOLOv8s model involves optimizing a multi-component loss function that combines localization, objectness, and classification objectives. This decoupled formulation enables more focused learning per prediction head and improves stability during training especially critical in biomedical tasks with class imbalance and small object scales.

The total loss function is defined as:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{box}} \mathcal{L}_{\text{box}} + \lambda_{\text{obj}} \mathcal{L}_{\text{obj}} + \lambda_{\text{cls}} \mathcal{L}_{\text{cls}}$$

Where:

- $\mathcal{L}_{\text{box}}$: bounding box regression loss

- $\mathcal{L}_{\text{obj}}$: objectness (binary classification) loss
- $\mathcal{L}_{\text{cls}}$: class prediction loss
- $\lambda_{\text{box}}, \lambda_{\text{obj}}, \lambda_{\text{cls}}$: loss balancing coefficients (empirically tuned)

1. Localization Loss – CIOU YOLOv8 uses the Complete Intersection over Union (CIOU) loss to measure the difference between predicted and ground truth bounding boxes [23]:

$$\mathcal{L}_{\text{CIOU}} = 1 - \text{IoU} + \frac{\rho^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2} + \alpha v$$

Where:

- IoU: overlap between predicted and true boxes
- ρ : Euclidean distance between box centers
- c : diagonal of the smallest enclosing box
- v : aspect ratio consistency term
- α : weight for aspect ratio penalty

2. Objectness Loss The objectness score is optimized using Binary Cross-Entropy (BCE) loss, which compares the predicted probability of object presence to the ground truth label (0 or 1) [22]:

$$\mathcal{L}_{\text{obj}} = -[y \log(p) + (1 - y) \log(1 - p)]$$

Where:

- y : ground truth (1 for object, 0 for background)
- p : predicted objectness probability

3. Classification Loss Each class prediction is treated as an independent binary classification task using sigmoid + BCE loss [22]:

$$\mathcal{L}_{\text{cls}} = \sum_{i=1}^C -[y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

Where:

- C : number of classes
- y_i : ground truth label for class i
- p_i : predicted probability for class i

All three loss components are calculated per detection point and averaged over the batch. In practice, the Ultralytics YOLOv8 framework uses default loss weights:

$$\lambda_{\text{box}} = 1.0, \quad \lambda_{\text{obj}} = 1.0, \quad \lambda_{\text{cls}} = 1.0$$

These weights were retained during our experiments due to consistent convergence and performance across classes.

4.2.3 Training Schedule and Techniques

Table 4.1: Technologies and Setup for YOLOv8s Pipeline

Component	Technology / Tool
Model	YOLOv8s (Ultralytics)
Framework	PyTorch (via Ultralytics wrapper)
Training Engine	Ultralytics CLI & Trainer API
Augmentations	mosaic, mixup, flips, jitter
Annotation Format	YOLO TXT format + data.yaml config
Data Tiling	Python (custom script using PIL, NumPy, OpenCV)
Oversampling	Manual duplication of tiles for minority classes
Evaluation	Built-in Ultralytics validation engine
Visualization	Matplotlib
Hardware	Google Colab Pro (A100 GPU)

To maximize model performance on a relatively small and imbalanced dataset, we implemented a combination of augmentation techniques and targeted oversampling. In this section we present the complete training setup for YOLOv8s, detailing the augmentation strategy and class balancing strategies

Augmentation during training As discussed in Sections 3.2 and 3.5, class imbalance and limited data were major challenges in this study. we depended on YOLOv8’s built in augmentation engine in hopes that we could overcome the limitations of the data and strengthen the model. Augmentations were applied in real time without changing the original dataset. The following parameters were selected based on experimental results and the traits of WSIs :

- **Mosaic (mosaic = 1.0):** four training images were combined into a single composite one, to boost the diversity by mimicking different object sizes and arrangements.
- **MixUp (mixup = 0.2):** two training images were blended together in a linear way with their labels, to make new in-between examples which will help the model learn better and avoid overfitting as well.
- **Geometric Transformations:**
 - *degrees = 5.0:* we apply minor rotations ± 5 degrees to simulate natural angular variations and preserve our structure at the same time.
 - *translate = 0.1:* Introduces 10% random shifts to handle positional inconsistencies.
 - *scale = 0.5:* Rescales by $\pm 50\%$ to simulate different magnification levels, very critical in medical imaging.
 - *shear = 2.0:* we applied minor shearing so we can replicate the microscopic slide deformation that can usually occur during slide preparation.

- $flipud = 0.5$, $fliplr = 0.5$: Vertical and horizontal flips with 50% probability, to imitate the various orientations that occur in medical slides while preserving the biological integrity as well

These transformations were applied carefully to preserve biological integrity and avoid the distortions that may occur from excessive shear or brightness shifts

Training Schedule and Optimization Training Schedule and Optimization we employed 200 epochs with a batch size of 8, to find the best balance between convergence stability alongside the constraints of GPU memory.

```
!yolo detect train
  model=yolov8s.yaml
  data=data.yaml
  imgsz=1024
  batch=8
  epochs=200
  lr0=0.001
  warmup_epochs=3
  hsv_h=0.015 hsv_s=0.7 hsv_v=0.4
  degrees=5 translate=0.1 scale=0.5 shear=2
  flipud=0.5 fliplr=0.5
  mosaic=1.0 mixup=0.2
```

Listing 4.1: YOLOv8s training command with augmentation and hyperparameter settings.

4.2.4 End-to-End YOLOv8s Pipeline and Technologies

In the following section, we will describe how the pipeline of yolov8s function from beginning to end, to summarize our key steps :

1. Data Acquisition and Preprocessing

- The Raw slides were retrieved in .tif format via SlideMaster, alongside their corresponding XML polygon annotations and Excel-based class metadata.
- Polygon annotations were then converted into bounding boxes, and each one of those boxes was mapped to its villus class using our preprocessed Excel sheet.

2. Tiling and Label Assignment

- Slides were tiled into 1024×1024 patches with 25% overlap.
- Each bounding box was shifted and adjusted to its local coordinates within the tile. Also , we only kept the tiles that had valid bounding boxes.
- The bounding boxes were then saved in YOLO format: (class_id, x_center, y_center, width, height).

3. Data Splitting and Oversampling

- we split the tiles randomly using a fixed seed into 80% training, 10% validation, 10% test.
- The oversampling was only applied to training tiles that contained Intermediate or Stem villi ($\times 3$ replication), the validation and test sets remained untouched.

4. Training Configuration

- Training was conducted for 200 epochs with a batch size of 8.
- The model used yolov8s.yaml architecture with input resolution of 1024×1024 .

5. Augmentation Strategy

- Augmentations were applied in real time during training through the Ultralytics engine:
 - * Mosaic (mosaic=1.0)
 - * MixUp (mixup=0.2)
 - * Geometric transforms: rotation (5°), translation (10%), scaling (50%), shearing (2%), and horizontal/vertical flips (50%).

6. Output and Evaluation

- Model predictions per tile include:
 - * its Objectness score: $\text{Pr}(\text{Object})$, Class probabilities(softmax over classes) and Bounding box (x, y, w, h)
- postprocessing includes confidence thresholding and Non-Maximum Suppression (NMS).
- The Best checkpoint saved based on its validation mAP.

4.3 Architecture and Model Configuration (Faster R-CNN)

The detection pipeline uses the Faster R-CNN framework with a ResNet-50 backbone and Feature Pyramid Network (FPN), selected for its accuracy in detecting small and complex structures like placental villi.

Backbone Architectures in Medical Object Detection

Convolutional Backbone and Feature Pyramid Faster R-CNN processes each 2048×2048 tile through ResNet-50 pretrained on ImageNet to extract the hierarchical features. We attach FPN to the backbone to promote further enhancement for multi-scale detection, This helps in identifying objects at different sizes by combining both high resolution and semantically rich low-resolution features

$$P_l = \text{FPN}(C_l)$$

where C_l is the feature from the backbone, and P_l is the output at level l in the pyramid.

[1]

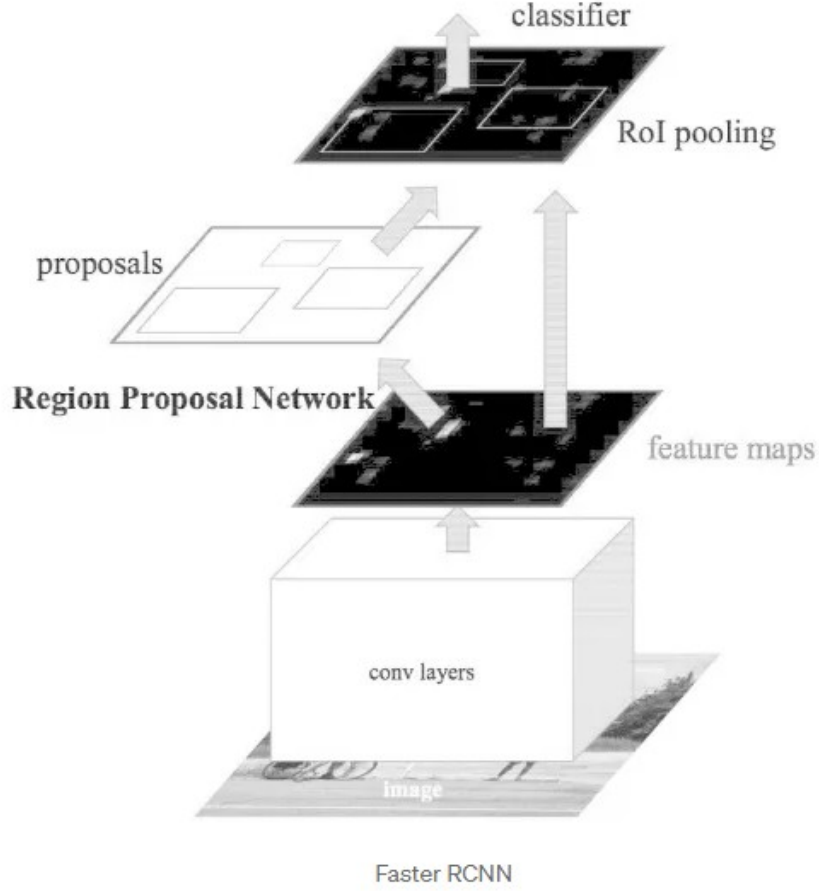


Figure 4.2: Two-stage detector.[31]

Region Proposal Network (RPN) Region Proposal Network (RPN) The RPN takes in the multi-scale feature maps extracted from the FPN and suggests a possible candidate object region using a sliding window approach. Each anchor box is evaluated with an objectness score and adjusted with regression offsets. The RPN optimizes the following loss:

$$\mathcal{L}_{\text{RPN}} = \mathcal{L}_{\text{cls}}(p_i, p_i^*) + \lambda \mathcal{L}_{\text{reg}}(t_i, t_i^*)$$

where p_i is the predicted objectness probability, p_i^* is the ground truth label, t_i is the predicted box parameter, and t_i^* is the ground truth offset.

[2]

ROI Align and Detection Head The top-scoring proposals from the RPN are passed through ROI Align, which extracts fixed-size features from variable-sized proposals. These are processed through two fully connected layers:

- **Softmax classifier:** Predicts class probabilities.
- **Bounding box regressor:** Refines localization via smooth L1 loss.

Bounding box transformations:

$$t_x = \frac{x - x_a}{w_a}, \quad t_y = \frac{y - y_a}{h_a}, \quad t_w = \log \frac{w}{w_a}, \quad t_h = \log \frac{h}{h_a}$$

Reference: [2]

Inference and Post-Processing After classification and regression, overlapping boxes are filtered using Non-Maximum Suppression (NMS) based on the predicted class probabilities (no objectness score product as in YOLO).

Model Configuration Summary

- **Backbone:** ResNet-50 + FPN
- **Pretrained:** COCO weights
- **Anchors:** [32, 64, 128, 256, 512]
- **Aspect Ratios:** [0.5, 1.0, 2.0]
- **ROI Output Size:** 7×7
- **Loss:** Cross-Entropy + Smooth L1
- **Tile Size:** 2048×2048
- **Number of Classes:** 3 (Terminal, Intermediate, Stem)

4.3.1 Forward Pass and Output Heads

As we had already established, Faster R-CNN follows a two-stage object detection pipeline. Each image tile passes through several stages in sequence: backbone feature extraction, region proposal generation, and region-wise classification and bounding box regression.

1. Feature Extraction

The input image is first passed through the ResNet-50 backbone, which was initialized with pre-trained ImageNet weights. This CNN transforms the image into a lower-resolution feature map, which contains rich hierarchical features representing edges, textures, and semantic structures. Let $I \in \mathbb{R}^{H \times W \times 3}$ be the input image, then the output after the backbone is:

$$F = \text{ResNet50}(I), \quad F \in \mathbb{R}^{H' \times W' \times C}$$

where H' , W' , and C are the height, width, and number of channels in the output feature map respectively. This dense feature map feeds into the Region Proposal Network (RPN). [27, 4]

2. Region Proposal Network (RPN)

The RPN scans the entire feature map F using sliding windows and proposes regions of interest (RoIs) that may contain objects. It does so by placing anchors of various scales and aspect ratios at each spatial location of the feature map. Each anchor is scored with:

- An objectness score $p_{\text{obj}} \in [0, 1]$, indicating if the anchor likely contains an object.

- Bounding box deltas (t_x, t_y, t_w, t_h) , which are offsets used to refine the anchor into a more precise bounding box.

The bounding box offsets are computed relative to the anchor coordinates (x_a, y_a, w_a, h_a) :

$$t_x = \frac{x - x_a}{w_a}, \quad t_y = \frac{y - y_a}{h_a}, \quad t_w = \log \frac{w}{w_a}, \quad t_h = \log \frac{h}{h_a},$$

where (x, y, w, h) are the predicted box parameters. Reference: [27, 2]

3. ROI Pooling

After the top- N proposals are selected using non-maximum suppression (NMS) based on their objectness scores, they are passed to ROI Pooling. Since the proposed boxes have varying sizes, ROI Pooling converts each one into a fixed-size feature map (e.g., $7 \times 7 \times C$), ensuring consistent input size for the final head. [27, 1]

4. Classification and Box Regression Head

Each ROI's fixed-size feature map is fed into two fully connected branches:

- A classifier that outputs a probability distribution $P(c \mid \text{ROI})$ over K classes + background.
- A bounding box regressor that predicts refined (t_x, t_y, t_w, t_h) for each class.

The total number of regression outputs becomes $4 \times (K + 1)$. These outputs are computed as:

$$\hat{p}_i = \text{softmax}(W_c \cdot f_i + b_c), \quad \hat{t}_i = W_r \cdot f_i + b_r,$$

where f_i is the pooled feature vector of the i th ROI, W_c, b_c are the weights and bias for the classification head, and W_r, b_r are the weights and bias for the regression head. [4, 2]

5. Final Prediction and NMS

From all proposed RoIs, the model selects final boxes based on class scores, and applies NMS again to suppress duplicate detections. The Intersection-over-Union (IoU) threshold for suppression is typically set between 0.3 and 0.7. Only boxes with high confidence scores are retained for the final output.

4.3.2 Loss Functions and Training Flow

Faster R-CNN deploys a joint-task loss function that simultaneously optimizes two goals: object classification and bounding box regression. The loss is calculated at two points in the network:

1. Region Proposal Network Loss The RPN loss is a combination of binary classification loss (object vs. background) and bounding box regression loss for anchors labeled as positive. Formally:

$$\mathcal{L}_{\text{RPN}} = \frac{1}{N_{\text{cls}}} \sum_i \mathcal{L}_{\text{cls}}(p_i, p_i^*) + \lambda \frac{1}{N_{\text{reg}}} \sum_i \mathbf{1}_{\{p_i^*=1\}} \mathcal{L}_{\text{reg}}(t_i, t_i^*),$$

where

- p_i is the predicted objectness score for anchor i , $p_i^* \in \{0, 1\}$ the ground truth label,

- t_i are the predicted box offsets, t_i^* the ground truth offsets,
- $\mathcal{L}_{\text{cls}}$ is binary cross-entropy loss,
- $\mathcal{L}_{\text{reg}}$ is Smooth L1 loss,
- λ is a balancing factor (typically set to 10).

[27, 2]

2. ROI Head Loss For the final predictions, the model computes:

$$\mathcal{L}_{\text{head}} = \mathcal{L}_{\text{cls}}(\hat{p}_i, c_i) + \mathbf{1}_{\{c_i \neq \text{bg}\}} \mathcal{L}_{\text{reg}}(\hat{t}_i(c_i), t_i^*),$$

where

- $\hat{p}_i$ are the predicted class probabilities, c_i the ground truth class,
- $\hat{t}_i(c_i)$ predicted box deltas for class c_i , t_i^* the ground truth deltas,
- $\mathcal{L}_{\text{cls}}$ cross-entropy loss,
- $\mathcal{L}_{\text{reg}}$ Smooth L1 loss.

Only ROIs labeled as foreground contribute to the regression loss. [27, 4, 2]

3. Total Loss The overall loss is the sum of RPN and Head losses:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{RPN}} + \mathcal{L}_{\text{head}}.$$

[27, 4, 2]

Training Flow in Practice In the implemented pipeline:

- **Optimizer:** Adam with a learning rate of 1e-4, applied only to trainable parameters.
- **LR Scheduler:** Step decay scheduler with $\gamma = 0.1$ applied every 10 epochs.
- **Training:** 30 epochs with loss components tracked per epoch. Skipped batches were also counted for logging purposes.
- **Validation:** Loss monitoring was done manually, with no automatic checkpointing shown in the snippet.
- **Monitoring:** Total loss and its components were logged to monitor convergence and stability.

4.3.3 Training Techniques and Augmentations

Training the Faster R-CNN model for placental villi detection required a carefully tailored strategy to balance data scarcity, class imbalance, and computational limitations. This section describes the core training methodology, including the augmentation pipeline, oversampling, and training configuration.

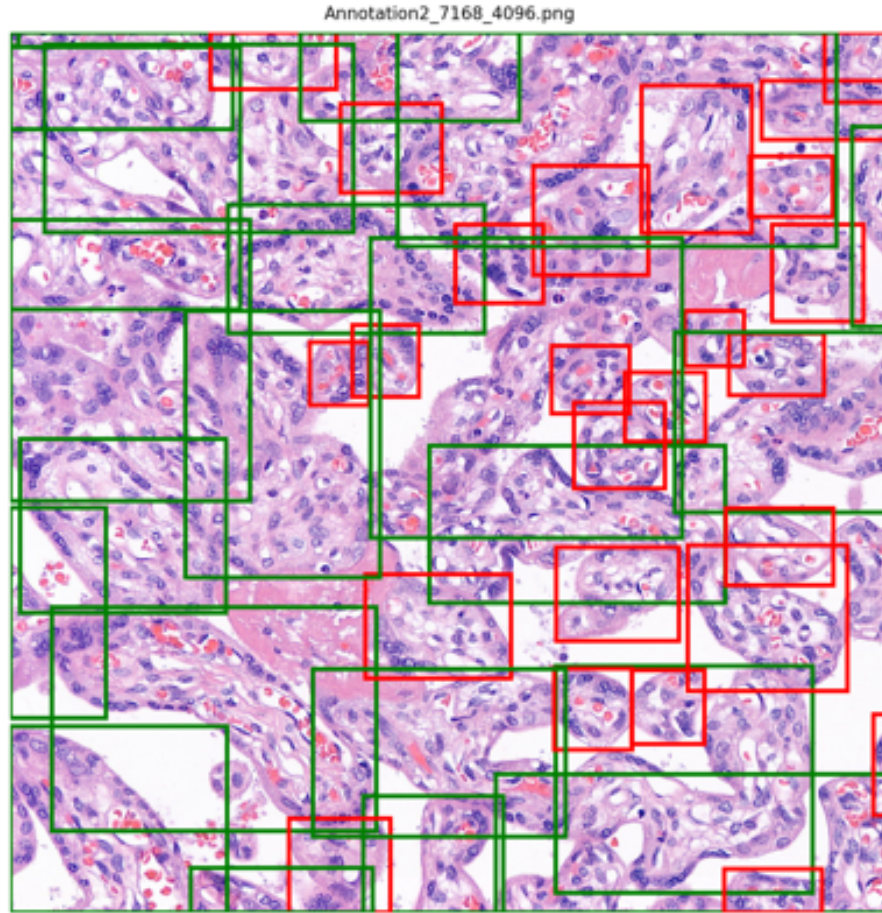


Figure 4.3: Sample annotations grid used for training

1. Data Augmentation Strategy Augmentations were through *PyTorch*'s *torchvision.transforms* library and applied only to the training set to avoid creating distortions to the validation and test results, so we can at the same time achieve data enrichment and preserve the biological integrity

```
from torchvision.transforms import Compose, ColorJitter, RandomHorizontalFlip,
    RandomVerticalFlip, ToTensor

train_transforms = Compose([
    RandomHorizontalFlip(),
    RandomVerticalFlip(),
    ColorJitter(brightness=0.2, contrast=0.2),
    ToTensor()
])
```

Listing 4.2: Augmentation transforms

These were selected based on the following rationale:

- **RandomHorizontalFlip / RandomVerticalFlip ($p = 0.5$):** Villi exhibit no directional bias, so flips simulate orientation variability.
- **ColorJitter:** Introduces variation in brightness and contrast, mimicking staining variability in real-world slides.

These types of augmentations are common in medical imaging due to their low risk of distorting semantic meaning

2. Oversampling of Rare Classes As described in Section 3.2.2, Stem villi (class 3) were severely underrepresented, while the intermediate villi were somewhat of a moderate quantity. To counteract this, tiles containing Intermediate (2) and Stem (3) villi were replicated 4× in the training set.

```
oversampled_tiles = []
for tile in train_tiles:
    labels = [ann['label'] for ann in tile['annotations']]
    if 2 in labels or 3 in labels:
        oversampled_tiles.extend([tile] * 4) #Oversample tiles with
        label2or3
    else:
        oversampled_tiles.append(tile)
```

Listing 4.3: Oversampling logic

This increased the exposure of minority classes during gradient updates and reduced bias toward the dominant Terminal villi class (1). Oversampling is a standard response to class imbalance and shown to improve recall on rare categories in object detection [11, 2].

3. Training Configuration The model was trained for 30 epochs using the Adam optimizer with a learning rate of 1×10^{-4} . The step scheduler helped to reduce the learning rate by 0.1 every 10 epochs. Also, Oversampling and augmentation helped balance underrepresented classes, and loss components were logged throughout training.

```
params = [p for p in model.parameters() if p.requires_grad]
optimizer = torch.optim.Adam(params, lr=1e-4)
lr_scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=10, gamma=0.1)
num_epochs = 30
for epoch in range(num_epochs):
    model.train()
    total_loss = 0.0
    loss_classifier = 0.0
    loss_box_reg = 0.0
    loss_objectness = 0.0
    loss_rpn_box_reg = 0.0
    skipped_batches = 0 # Keep track for logging
```

Listing 4.4: Optimizer, Scheduler, and Training Loop

Loss values for each component classifier, box regression, objectness, and RPN were accumulated across batches. Batches with invalid labels or incorrect types were automatically skipped and logged.

4. Practical Notes

- All augmentation and oversampling were applied before training and saved in a .json format.

- Each training tile included image-level information with `image_id`, `boxes`, and `labels`, formatted for compatibility with PyTorch's `collate_fn`.
- No augmentations were applied during validation or inference.
- The model's output head was adapted to handle three foreground classes.

4.3.4 Inference Flow and Postprocessing

After training, the Faster RCNN model was used to generate predictions on test tiles. We will explain how the inference strategy is implemented, how raw predictions are processed, and how final outputs are generated and validated.

1. Model Evaluation Mode The model was switched to evaluation mode using:

```
model.eval()
```

This disables dropout layers and batch-norm updates, keeping predictions consistent.

2. Input Preparation Each input tile is loaded, normalized, and transformed similarly to training, but without augmentation:

```
image = Image.open(tile_path).convert("RGB")
image_tensor = transform(image).to(device).unsqueeze(0)
with torch.no_grad():
    prediction = model(image_tensor)[0]
```

3. Output Interpretation The prediction dictionary contains:

- `boxes`: predicted bounding boxes `[x1,y1,x2,y2]`
- `labels`: predicted class IDs (1 = Terminal, 2 = Intermediate, 3 = Stem)
- `scores`: confidence scores for each box (0 to 1)

A filtering threshold is applied:

```
boxes = prediction['boxes'].cpu().numpy()
labels = prediction['labels'].cpu().numpy()
scores = prediction['scores'].cpu().numpy()

for box, label, score in zip(boxes, labels, scores):
    if score < 0.5:
        continue
    xmin, ymin, xmax, ymax = box
    width, height = xmax - xmin, ymax - ymin
    color = label_colors.get(label, 'yellow')
    rect = patches.Rectangle((xmin, ymin), width, height,
                             linewidth=2, edgecolor=color, facecolor='none')
    ax.add_patch(rect)
    ax.text(xmin, ymin - 4, f"{label_names[label]} ({score:.2f})",
            color='white', fontsize=8, backgroundcolor='white')
```

4. Non-Maximum Suppression (NMS) PyTorch’s Faster R-CNN already includes NMS in its inference pipeline. It suppresses overlapping predictions using:

$$\text{IoU}(P_1, P_2) > \text{threshold} \implies \text{keep box with highest score}$$

Default threshold: 0.5

5. Visualization of Predictions To verify predictions visually, bounding boxes and labels were drawn over the tiles:

```
import matplotlib.pyplot as plt
import cv2

image_np = np.array(image)
for box, label in zip(boxes, labels):
    x1, y1, x2, y2 = box.astype(int)
    color = class_color_map[label]
    cv2.rectangle(image_np, (x1, y1), (x2, y2), color, 2)
    cv2.putText(image_np, class_names[label], (x1, y1 - 10),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 2)
plt.imshow(image_np)
```

These overlays were saved for human inspection to analyze false positives/negatives and class-wise errors.

6. Final Output Format Predictions were saved in JSON format per tile, including:

- Tile name
- Bounding box coordinates
- Predicted class
- Confidence score

This format makes it very easy to carry out work in later steps for further biological analysis.

Table 4.2: Technologies and Setup for Faster R-CNN Pipeline

Component	Technology / Tool
Model	Faster R-CNN with ResNet50-FPN backbone
Framework	PyTorch (torchvision.models.detection)
Training Engine	Custom PyTorch training loop
Augmentations	torchvision.transforms (ColorJitter, flips)
Annotation Format	JSON with bounding boxes + class IDs
Data Tiling	Custom Python script using PIL, NumPy
Oversampling	Manual 4× duplication for classes Intermediate and Stem
Loss Functions	Built-in (classification + Smooth L1 for bbox regression)
Inference Postprocessing	Built-in NMS via torchvision
Visualization	Matplotlib
Hardware	Google Colab Pro (A100 GPU)

4.3.5 Pipeline Summary – Faster R-CNN

The Faster R-CNN pipeline for placental villi detection was designed to accommodate the high-resolution nature of whole slide images (WSIs) and the need for precise localization of small and overlapping biological structures. The full workflow is summarized as follows:

1. Image Acquisition and Annotation Extraction

- Downloaded high-resolution slides (Annotation1 & Annotation2) in TIFF format from the Semmelweis SlideMaster platform.
- Parsed expert-annotated XML files using `xml.etree.ElementTree`, extracting polygon coordinates of villi.
- Used an Excel sheet mapping annotation names to class labels (Terminal, Intermediate, Stem) to match each polygon with its villus type.

2. Preprocessing and Annotation Conversion

- Converted polygons into axis-aligned bounding boxes in `(min_x, min_y, max_x, max_y)` format.
- Paired each bounding box with a numeric class label (1, 2, or 3).
- Stored annotations in a single JSON file compatible with PyTorch’s detection dataset loader.

3. Image Tiling

- Divided each slide into 2048×2048 overlapping tiles with a 50% stride (1024 pixels).
- Shifted and adjusted the bounding boxes in each tile to local coordinates.
- kept only the tiles that contained at least one valid object.
- Saved tile images in PNG format to preserve resolution and prevent lossy compression.

4. Data Augmentation and Oversampling

- Applied augmentations using `torchvision.transforms`: random horizontal/vertical flips, color jitter (brightness, contrast, saturation).
- Addressed class imbalance by duplicating tiles containing Intermediate or Stem villi $4\times$ in the training set.

5. Dataset Splitting

- Used a 70/15/15 split for training, validation, and test sets, respectively.
- Shuffled tiles using a fixed seed so we can guarantee non-overlapping splits.
- Passed annotations and image paths to a custom PyTorch Dataset class.

6. Model Configuration and Training

- Employed `torchvision.models.detection.fasterrcnn_resnet50_fpn` with pretrained weights.
- Training parameters:
 - * Epochs: 30
 - * Batch size: 4
 - * Optimizer: Adam with learning rate 1×10^{-4}
 - * Learning rate scheduler: `tepLR` (step size = 10, gamma = 0.1)
- a custom training loop was employed to log losses and evaluation metrics each epoch.

7. Inference and Postprocessing

- we ran inference on test tiles to generate bounding boxes, scores, and labels.
- Filtered predictions with:
 - * A confidence threshold (0.5)
 - * Non-Maximum Suppression (NMS) to remove overlapping detections
- Visualized final results using Matplotlib overlays on each tile.

8. Output Handling and Evaluation

- Saved predictions per tile in JSON format, including tile name, predicted label, bounding box coordinates, and confidence scores.
- Computed evaluation metrics (mAP, precision, recall) on the test set through IoU thresholds.

We had designed this pipeline with the idea to leverage the two-stage nature of Faster R-CNN, to guarantee strong region proposals and accurate classification. The full training and evaluation cycle was performed in Google Colab Pro with an A100 GPU, using native PyTorch tools and custom augmentation and preprocessing scripts.

4.4 Postprocessing

Following inference on tiled patches of the original high-resolution images , we applied a series of postprocessing steps to merge those tiles back to a unified image-wide predictions. These techniques were implemented on

Coordinate remapping: adjusted the bounding box coordinates from local tile coordinates back to the global coordinates of the original .tif image by using the stored tile offsets .

Non-Maximum Suppression (NMS) : a technique used to reduce the overlapping predictions , where the algorithm will select the most confident box of each object and suppresses the others based on a predefined IoU threshold .[7]

4.4.1 Postprocessing Pipeline for YOLOv8s Inference

As a summary of the previous step, detection was performed by running inference on 1024×1024 tiles with 25% overlap. Each tile was processed independently, and the predictions were saved in the required .txt YOLO format using normalized coordinates.

Tiling and Metadata Generation In the preprocessing step , we had saved the position of each tile in a dictionary to facilitate the reconstruction of the global predictions

```
tile_info[tile_id] = {
    "x_offset": x,
    "y_offset": y,
    "width": right - x,
    "height": bottom - y
}
```

Coordinate Remapping

When the inference was done , every tile corresponding predictions were parsed and denormalized. The bounding boxes were then translated into global image coordinates using the tile's offset:

```
x1 = x_center - width / 2 + x_offset
y1 = y_center - height / 2 + y_offset
x2 = x_center + width / 2 + x_offset
y2 = y_center + height / 2 + y_offset
```

which resulted in generating a complete set of bounding boxes for the full resolution image .

Non-Maximum Suppression (NMS) Due to the tiles overlap, some objects were detected multiple times across adjacent patches. NMS was applied to remove these duplicates , selecting only the highest confidence box per object and supressing the rest with IoU overlap beyond a threshold.

we implemented class-wise NMS using PyTorch's torchvision.ops.nms. We had to do trials over several thresholds to see which were giving the best outlook :

IoU = 0.5 (default): had too many overlapping boxes.

IoU = 0.4: gave the cleanest results without excessive running even though there were still some misplaced boxes.

IoU = 0.3: too strict which had led to visibly missed detections.

```
keep = torchvision.ops.nms(boxes, scores, iou_threshold=0.4)
filtered_preds = pred_tensor[keep]
```

4.4.2 Postprocessing Pipeline for FasterRcnn Inference

To run detection on full-resolution placental tissue analysis slides using *Faster R-CNN*, we used a patch-wise inference approach. Because the original images are very large and GPU memory is limited, we performed inference on overlapping tiles and then applied postprocessing to merge and clean up the predictions.

Tiling and Metadata Construction The .tif images were split into 2048×2048 tiles with 50% overlap . This tile size is larger than the one used for YOLOv8s (1024×1024), helping retain more visual context especially useful for detecting villi that are large or not clearly defined. Each tile was converted into a PyTorch tensor and stored along with its (*x_offset*, *y_offset*) to keep track of its position in the full image.

```
tiles_for_inference.append({
    'offset': (left, top),
    'crop_box': (left, top, right, bottom),
    'tensor': tile_tensor
})
```

Model Loading with Threshold Tuning

For this step , I had reloaded the trained Faster R-CNN model and customized a few inference parameters:

score_thresh = was at 0.5 and set to filter out low confidence predictions.

nms_thresh = was 0.3 and it was used to reduce any overlapping boxes during tile level detection.

These thresholds were manually set before running inference, which gave me more control compared to YOLOv8s where such tuning is more limited.

```
model.roi_heads.score_thresh = 0.5
model.roi_heads.nms_thresh = 0.3
```

Inference and Global Coordinate Remapping

For each object the model detects in a tile, it gives a bounding box based on that tile's local coordinates. This means:

- The top-left corner of the tile is (0,0).
- The box coordinates are measured from that point.

But to put together predictions from all tiles into the full image, we need to shift those boxes into the global coordinate system.

We do this by adding the tile's (x, y) offset its position in the original .tif image to each box coordinate.

```
global_box = [
    xmin + offset_x, # move left edge to global position
    ymin + offset_y, # move top edge to global position
    xmax + offset_x, # move right edge to global position
    ymax + offset_y  # move bottom edge to global position
]
```

The bounding box is moved from the tile's local coordinates to its correct spot in the original full-resolution image.

To keep things organized for the next steps, like NMS, we saved the predictions in three separate lists (practically the same approach as in yolov8s):

- all_boxes: the bounding boxes, now shifted to match the full image.
- all_scores: the confidence score for each detection.
- all_labels: the class ID for each prediction (1 = Terminal, 2 = Intermediate, 3 = Stem villi).

These lists were later passed into the global NMS step.

To shift the boxes correctly, we simply added the tile's (x, y) offset its location within the original .tif image to each coordinate of the box.

Global Class-wise Non-Maximum Suppression (NMS) Similarly to what we saw in yolov8s the use of overlapping tiles led to the detection of the same object multiple times in adjacent patches. The NMS as we know is the go to fix . We used PyTorch's torchvision.ops.nms separately for each class, with an adjustable IoU threshold. This step was important to remove duplicate boxes and keep only the best prediction for each object.

Aggressive NMS Tuning

To reduce overlapping detections and make results clearer, we tested lower IoU thresholds for NMS:

0.3: Default some duplicates remained

0.2: there was Less overlap

0.15: it presented Good balance between removal and keeping valid boxes

0.1: Very strict only top detections kept in crowded areas

This tuning reduced the number of final boxes from over 1000 to around 300–800, depending on the class. It helped make the results easier to understand and analyze.

Visualization

Final detections were drawn on the full Annotation2.tif image using:

Red for Terminal villi , Green for Intermediate , Blue for Stem

Feature	YOLOv8s	Faster R-CNN
Tile size	1024×1024	2048×2048
Tile overlap	256 px (25%)	1024 px (50%)
Remapping method	Denormalized YOLO format	Absolute pixel coordinates
Confidence threshold	0.25	0.5
Built-in NMS	Yes, during tile-level inference	Yes, with manual global control
Global NMS applied manually	Yes (post-inference)	Yes (class-wise, post-inference)
IoU tuning	0.5 → 0.4	0.3 → 0.2 → 0.15 → 0.1
Final box count (after NMS)	Higher, less filtered	Lower, more precise (300–800 boxes)
Visual quality	Cluttered initially	Clearer and cleaner after tuning

Table 4.3: Postprocessing for YOLOv8s vs Faster R-CNN.

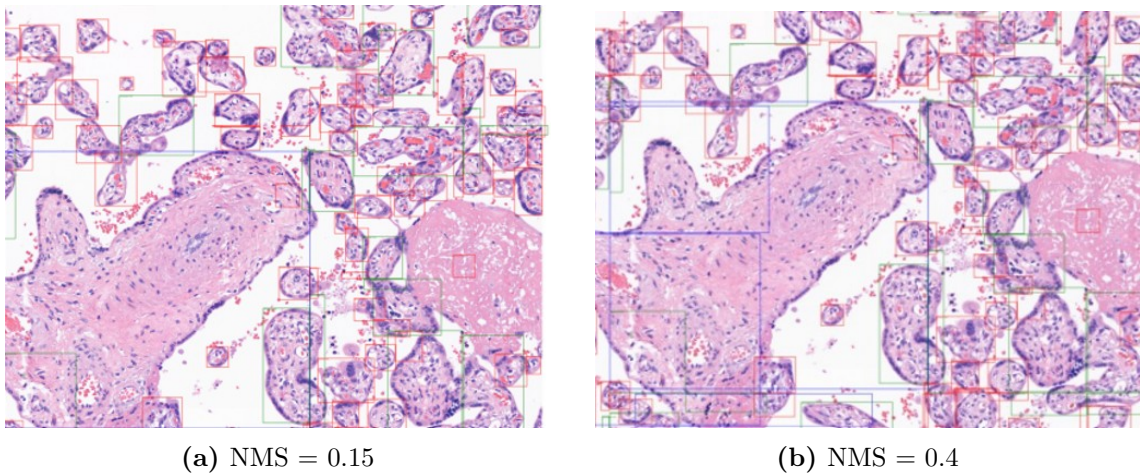


Figure 4.4: Comparison of two Faster R-CNN outputs with different NMS thresholds for a closer shot clarity

This side-by-side comparison shows how the NMS threshold affects how the overlapping boxes are handled and not whether objects are detected . At **NMS = 0.15**, the model produces a very clean view by strictly suppressing and removing overlaps, but this can result in missed detections in dense regions. With **NMS = 0.4**, a lot more overlaps were kept, which means the coverage may be good, but results in the output being visually busier. Choosing the right NMS is a trade-off

between clarity and completeness, which is especially important when villi are tightly clustered.

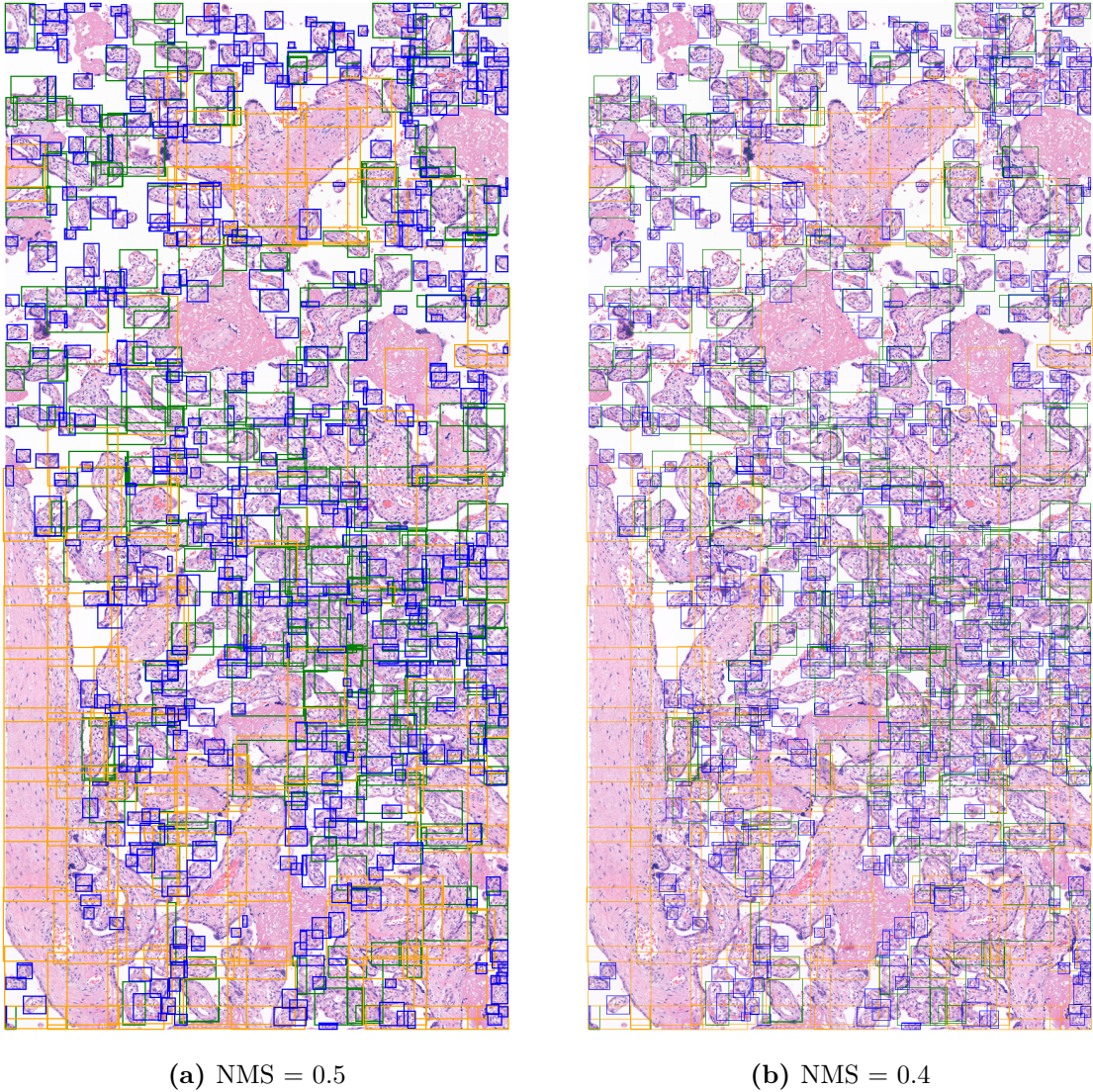


Figure 4.5: Comparison of two YOLOv8 outputs with different NMS thresholds for a wider shot clarity

The comparison between $\text{NMS} = 0.5$ (left) and $\text{NMS} = 0.4$ (right) illustrates how adjusting the suppression threshold changes the way YOLOv8s handles overlapping detections. With NMS at 0.5, the model has more coverage, meaning it keeps more boxes, which unfortunately leads to visual clutter, while NMS at 0.4 is stricter and filters out more overlaps, generating a cleaner output and is easier to interpret. Still, in both cases, we notice that the larger villi have several smaller bounding boxes. This doesn't mean that detections were missed; instead, it reflects how complex the internal structure of large villi can be. Features inside them may seem to be different, smaller, and separate villi to the model. The tiling strategy might also cut across these structures, breaking their context. On top of that, the model may have gotten used to seeing smaller villi more often during training, which could explain the tendency to over-segment. While not ideal, this behavior is promising; it shows

that the model is picking up on fine-grained patterns and could improve with better guidance, like more structured labels.

Chapter 5

Evaluation

5.1 Evaluation Metrics and Loss Functions

In medical object detection, specifically in histological tasks, it is not enough to look at visual predictions. The model evaluation needs to be based on proper metrics to understand how the model is doing. These metrics must assess the localization and classification, meaning how well the object is accurately positioned and identified. Core tools for this evaluation include confusion matrix, precision, recall, F1-score, Intersection over Union (IoU) and mean Average Precision (mAP). These metrics help measure real performance while loss functions guide learning and improvement throughout training [23].

5.1.1 Confusion Matrix and Core Metrics

It summarizes the fundamental detection outcomes:

- **True Positive (TP):** Correct prediction of an object with IoU above threshold.
- **False Positive (FP):** Predicted box is incorrect, either wrong class or low IoU.
- **False Negative (FN):** Ground truth object was missed (no box predicted).
- **True Negative (TN):** Correct identification of regions with no objects.

From that, we derive precision, recall, and F1 score:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad \text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

If the model has high precision but low recall (or vice versa), the F1 score will be low; if both are good, F1 will be high. These are very important when dealing with class imbalance [24].

5.1.2 Intersection over Union (IoU)

- Measures overlap between predicted bounding boxes and ground truth.
- Central to computing precision, recall, and mAP [4].
- Considered correct if IoU exceeds a threshold.

The IoU is computed as:

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}.$$

5.1.3 Mean Average Precision (mAP)

- *Average Precision (AP)*: Measures the area under the precision–recall curve for a class.
- *Mean Average Precision (mAP)*: Mean of AP values over all classes.
- mAP@0.5: AP at IoU = 0.5.
- mAP@[0.5 : 0.95]: COCO-style metric averaging AP over IoU thresholds from 0.5 to 0.95 for comprehensive evaluation [24].

In medical imaging, especially with small, overlapping structures, mAP@[0.5 : 0.95] is more indicative of subtle performance.

5.1.4 Loss Functions in Object Detection Models

Loss functions guide training and quantify error by combining classification, localization, and objectness terms.

- **YOLOv8**: Uses a compound loss including classification loss (cross-entropy), localization loss (CIoU/GIoU), and objectness loss to ensure accurate presence and positioning of objects.
- **Faster R-CNN**: Computes total loss as the sum of classification loss (cross-entropy) and localization loss (Smooth L1) over both the Region Proposal Network (RPN) and the final detection head.

5.2 Yolov8s Evaluation

5.2.1 Sample Test Tile Visualizations

Before we delve into the evaluation metrics, we show a sample of test tiles that illustrate the bounding box predictions of the YOLOv8s model. These visualizations provide an understanding of how the model performs on test data, particularly in terms of detection accuracy and class differentiation.

Insight: Terminal villi are detected with high confidence and correct placement of bounding boxes, but it comes with slightly more variability and missed detections in the case of Intermediate and Stem villi. Some false positives are also visible, especially in regions with fibrotic or ambiguous tissue, where boundaries are less defined.

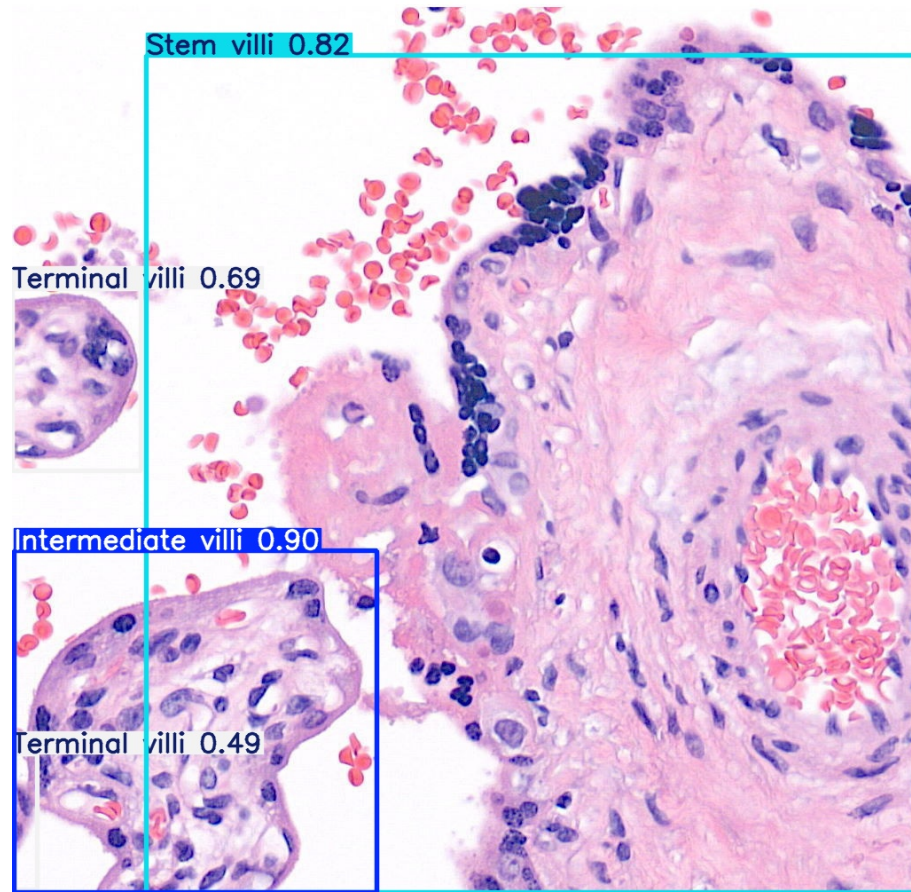


Figure 5.1: Sample test tiles with YOLOv8s bounding box predictions

Analysis: The model successfully identifies all three villus types within this given tile: terminal villi (confidence: 0.69, 0.49), Intermediate villi (0.90), and Stem villi (0.82). Bounding boxes are color-coded specifically to the class and labeled with confidence scores. This detection highlights the work done and the strategies used during the training so that the model can localize and classify multiple villi types within a histological tile despite class imbalance and visual overlap.

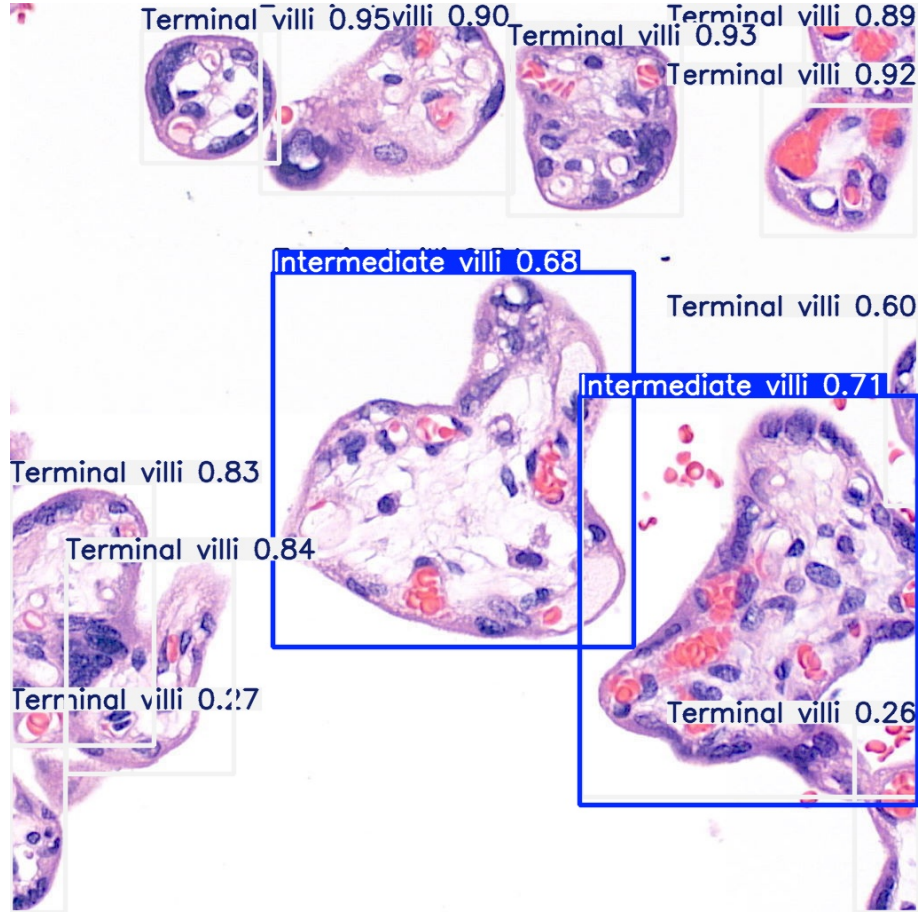


Figure 5.2: Sample test tiles with YOLOv8s bounding box predictions

Analysis : Terminal villi are detected with high confidence as in most cases (up to 0.95), though some boxes show a confidence score of 0.26, 0.27, indicating reduced certainty in visually complex regions. Intermediate villi are successfully identified but exhibit minor boundary shift , meaning a minor localization error under dense tissue conditions.

5.2.2 Inference and Test-Time Evaluation

After training the YOLOv8s model, we evaluated its performance on a held-out test set consisting of 39 tiles. Inference was carried out with a confidence threshold of 0.25 and an IoU threshold of 0.5. The predictions were saved as both visual annotations and .txt files. For metric computation, we created a temporary data_test.yaml file and ran YOLOv8's built-in .val() function. The process yielded visual and statistical outputs, including confusion matrices, precision-recall curves, and class-wise metric plots. All results were then saved under runs/detect/val_results_plot/.

```

# Create a temporary data_test.yaml file pointing to the test split
data_test_yaml = "/content/yolo_dataset/data_test.yaml"
with open(data_test_yaml, "w") as f:
    f.write(f"""
path: /content/yolo_dataset
train: images/train
val: images/test

names: {list(class_to_id.keys())}
""")

# Evaluate model on the test set
results = model.val(
    data=data_test_yaml,
    save_json=True,
    conf=0.25
)

```

Listing 5.1: YOLOv8s Test-Time Evaluation Script

5.2.3 Quantitative Results

Quantitative evaluation of the YOLOv8s model on the given test set depended on using the standard object detection metrics, including Precision, Recall, mAP@0.5, and mAP@[0.5 : 0.95]. These metrics were computed both per class and overall aggregates. Performance curves such as precision–recall, F1–confidence, and confusion matrices were also generated to visualize classification performance and threshold effects.

Table 5.1: YOLOv8s Test Set Quantitative Results

Class	Precision	Recall	mAP@0.5	mAP@[0.5 : 0.95]
Terminal villi	0.752	0.794	0.819	0.707
Intermediate villi	0.694	0.662	0.694	0.537
Stem villi	0.556	0.577	0.596	0.337
Overall	0.667	0.678	0.703	0.527

Interpretation

- **Terminal villi:** As expected, showed strong performance across all metrics, indicating both accurate classification and consistent localization.
- **Intermediate villi:** Demonstrated moderate precision and recall but with a minor drop in mAP@[0.5 : 0.95], meaning reduced localization precision at stricter IoU thresholds.
- **Stem villi:** Had the weakest scores across all metrics, probably due to its scarcity, the huge class imbalance in the dataset, and its visual complexity (overlap with background or other classes).
- **Overall:** The model achieves mAP@0.5 of 0.703, demonstrating that YOLOv8s is well suited for villi detection, though performance varies and is affected by class-specific limitations.

5.2.4 Qualitative Results

This section highlights the model’s behaviour during training and validation. We analyze loss curves, mAP, confusion matrix, and precision/recall curves to spot if there were misidentification or overfitting issues. This will give us a summary of the model’s behaviour beyond the final scores.

5.2.4.1 Training and Validation Metrics Over Epochs

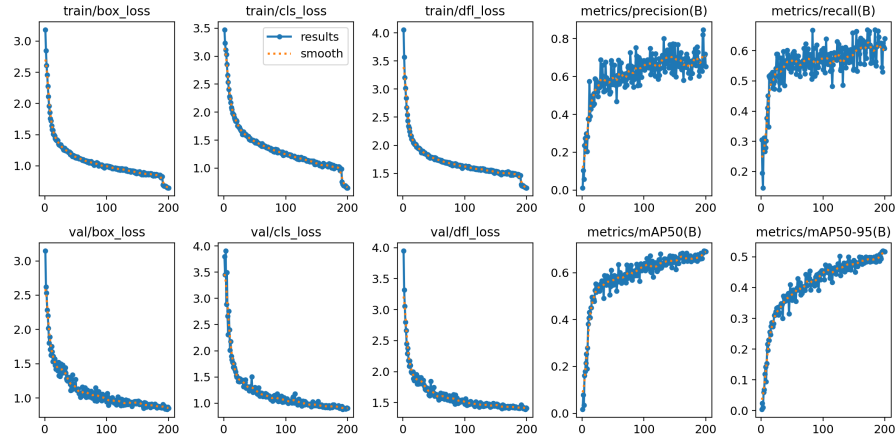


Figure 5.3: Training and Validation dynamics

Analysis: The classification, box, and distribution losses can be seen consistently dropping over 200 epochs, indicating good convergence. On the metrics side, both precision and recall also improved steadily, with the precision surpassing 0.8 by the end. The mAP curves for both mAP@0.5 and mAP@0.5:0.95 kept an upward rise, which settled near 0.68 and 0.52, respectively. the fluctuations in the PR curves are foreseen due to input diversity and class imbalance. The variations in YOLOv8’s per-epoch performance result from the infrequent occurrences of the rare classes (stem vill), and confidence threshold values. Overall, the trends reflect stable learning behavior with no signs of overfitting or divergence.

5.2.4.2 Confusion Matrix

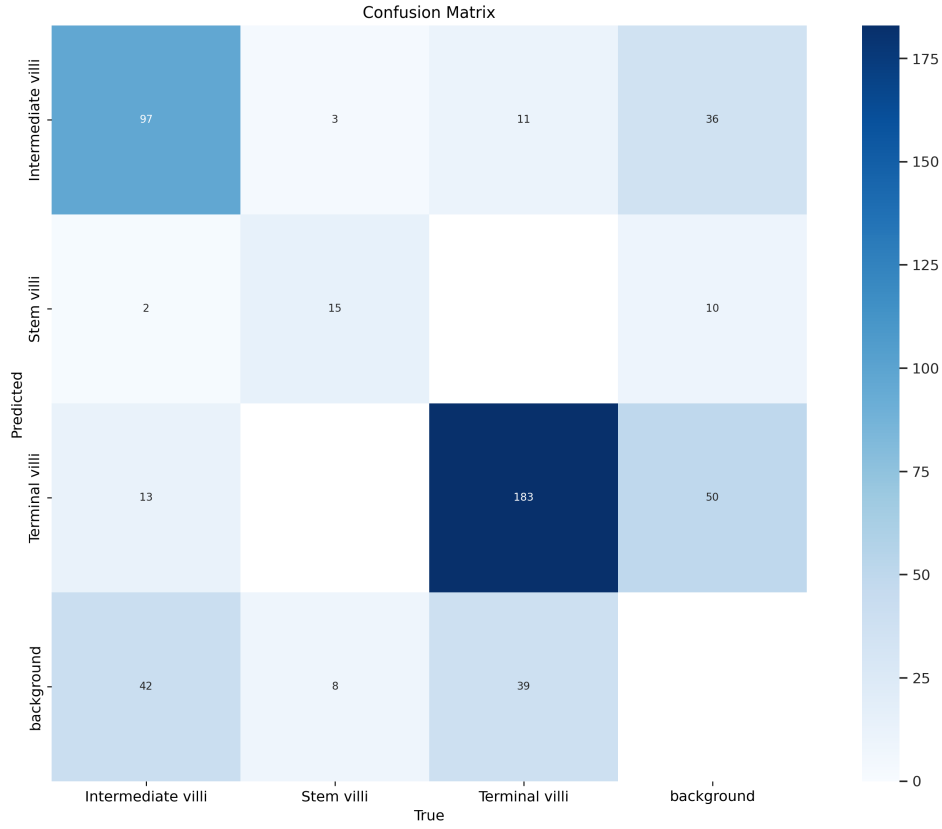


Figure 5.4: Confusion matrix of YOLOv8s predictions on the test set

Analysis: The model shows accurate detection for Terminal villi, with 183 True Positives (TP) out of 246. However, there is still confusion as we have 50 False Positives (FP) because the background was misclassified as Terminal, and 13 instances were confused with Intermediate villi. For the actual Intermediate villi, we see that the model achieves 97 True Positives out of 147. But at the same time, 36 FN were misclassified as background, and 11 were wrongly labeled as Terminal (FP). Stem villi remain the least accurate class, with only 15 TP out of 27, and 10 FN confused with background. The background class, which should have zero predictions as villi, shows some False Positives: 42 predicted as Intermediate, 8 as Stem, and 39 as Terminal. This points to the model's behavior to overpredict villi in empty regions. This points to the need for stronger background suppression or enhanced negative sampling during training.

5.2.4.3 Conclusion

Despite the stable training and good performance, YOLOv8 struggles when it comes to rare and limited classes (Stem villi in our case). These kinds of limitations, in precision and class separation, highlight the weaknesses when compared to two-stage detector models.

5.3 Faster R-CNN Evaluation

5.3.1 Visual Prediction Examples on Test Tiles

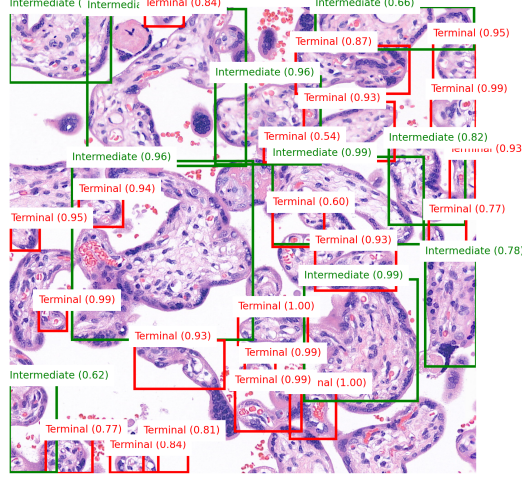


Figure 5.5: Example Faster R-CNN predictions on a 2048×2048 test tile.

We begin by inspecting test tiles to visually assess our model functionality before diving into metrics. These examples illustrate the model’s ability to localize and classify villi types. In comparison to YOLOv8s, the Faster R-CNN model was trained on larger tiles (2048×2048) as mentioned in Section 3.4. This made it more likely to encounter multiple, smaller villi in one view; therefore, overlaying predicted and true boxes side-by-side offers us clearer insights into detection quality and localization accuracy in dense cellular regions.

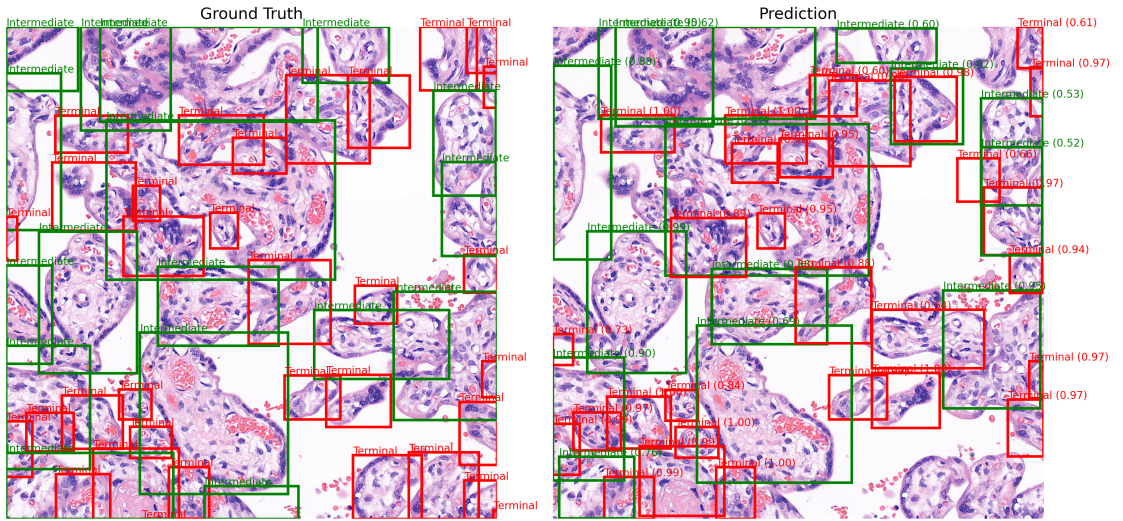


Figure 5.6: Faster R-CNN predictions (left) versus ground truth annotations (right) on a 2048×2048 test tile.

Analysis: The model successfully identifies Terminal and Intermediate villi with confidence higher than 0.95. However, overlapping instances result in duplicate boxes and missed detections. On a positive note, this highlights strong detection performance under dense conditions, but it comes with some limitations for close-proximity objects.

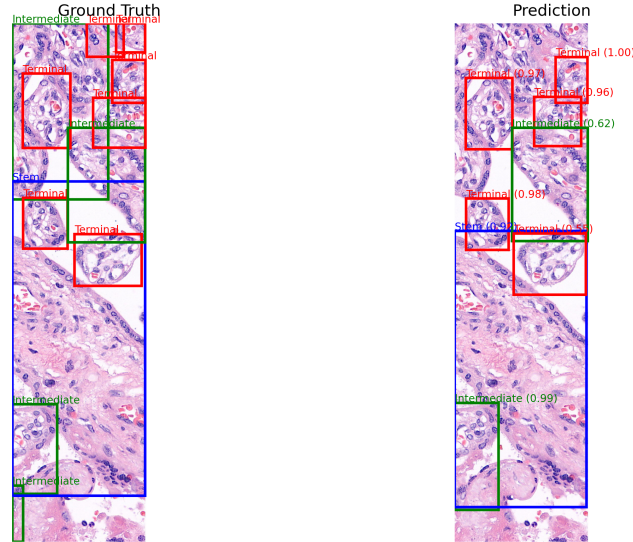


Figure 5.7: Faster R-CNN predictions (left) versus ground truth annotations (right) on a 2048×2048 test tile.

Analysis: This is still consistent with Figure 5.2; the model detects Terminal and Intermediate villi with a high confidence of 0.96. There are minor shifts that can be seen for Stem villi localization, confirming our earlier notions about the identification of less frequent classes under crowded and fine-grained scenarios.

5.3.2 Test Tile Inference and Visualization

Setup and model loading

```
save_dir = "predictions_all"
os.makedirs(save_dir, exist_ok=True)

model = fasterrcnn_resnet50_fpn(pretrained=False)
in_features = model.roi_heads.box_predictor.cls_score.in_features
model.roi_heads.box_predictor = torchvision.models.detection.faster_rcnn.
    FastRCNNPredictor(in_features, 4)
model.load_state_dict(torch.load('/content/fasterrcnn_villi.pth',
    map_location='cpu'))
model.to(device)
model.eval()
```

Listing 5.2: Loading the trained Faster R-CNN model and preparing for inference.

We start by reloading the model , We had restored the weight and the model has been switched to evaluation mode , from there we proceed to move on to the label configuration where we define class to color and class to name mapping for the visual annotation . (1: red, Terminal, 2: green, intermediate, 3: blue, stem)

Inference loop over test tiles

```
for idx in range(len(test_dataset)):
    image, _ = test_dataset[idx]
    image_gpu = image.to(device)

    with torch.no_grad():
        prediction = model([image_gpu])[0]
```

Listing 5.3: Iterating over the test set and running inference on each image.

For each of the 33 test tiles, we generated their corresponding predictions using matplotlib. We had disabled the gradient tracking to improve the speed and save memory as well. The model gave an output of bounding boxes, class labels, and confidence scores.

Draw Predicted Boxes

```
boxes = prediction['boxes'].cpu().numpy()
labels = prediction['labels'].cpu().numpy()
scores = prediction['scores'].cpu().numpy()

for box, label, score in zip(boxes, labels, scores):
    if score < 0.5:
        continue
    xmin, ymin, xmax, ymax = box
    color = label_colors.get(label, 'yellow')
    rect = patches.Rectangle((xmin, ymin), xmax - xmin, ymax - ymin,
                             linewidth=2, edgecolor=color, facecolor='none')
    ax.add_patch(rect)
    ax.text(xmin, ymin - 4, f"{label_names[label]} ({score:.2f})",
            color=color, fontsize=8, backgroundcolor='white')
```

Listing 5.4: Draw predicted boxes

Here as shown, we had applied a post-inference filter, so we take into account only the detections with confidence above 0.5. The boxes are then plotted with their corresponding confidence score and class-specific colors. The visual predictions were then saved to the predictions_all folder.

5.3.3 Quantitative Results

Table 5.2: Faster R-CNN test set quantitative results

Class	Precision	Recall	mAP@0.5	mAP@[0.5 : 0.95]
Terminal villi	0.7294	0.6764	0.700	0.490
Intermediate villi	0.6713	0.5555	0.610	0.410
Stem villi	0.4210	0.4571	0.440	0.280
Overall	0.6072	0.5630	0.583	0.393

We evaluated the Faster R-CNN model using the standard object detection metrics: **Precision**, **Recall**, **mAP@0.5**, and **mAP@[0.5:0.95]**.

Interpretation

Terminal villi

The model did well with Terminal villi, scoring a precision of 0.7294 and mAP@0.5 of 0.700. Although the recall was slightly lower at 0.6764, it still shows good localization. However, the mAP@[0.5:0.95] dropped to 0.490, which means the model struggled more at higher IoU thresholds.

Intermediate villi

This class had a precision of 0.671 and a recall of 0.555. On the other hand, the mAP@[0.5:0.95] was only 0.410, suggesting the model had trouble drawing precise boxes, probably because these villi have more shape variation or are sometimes partially hidden.

Stem villi

Stem villi were the hardest to detect. The model got the lowest precision (0.4210) and lowest mAP scores (0.440 at 0.5 IoU, and 0.280 at stricter IoU thresholds). This reflects how class imbalance and too few training samples made it harder for the model to learn this class well.

5.3.4 Qualitative Results

This section examines the evaluation behavior of the Faster RCNN model. We will analyze mAP scores, confusion matrix, and F1 vs Recall to assess how well the model can distinguish the villi classes.

5.3.4.1 Confusion Matrix

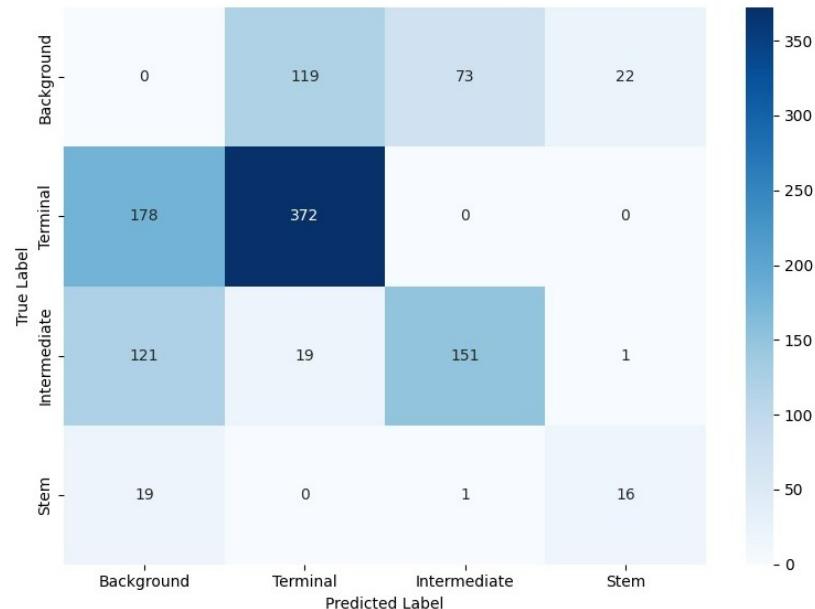


Figure 5.8: Confusion matrix of Faster R-CNN

The model performs well on **Terminal villi**, with 372 correct detections (True Positives). This shows how it's quite reliable at identifying this class. However,

there were still 178 missed detections, where Terminal villi were wrongly classified as background (False Negatives), meaning some were not picked up at all.

When it came to **Intermediate villi**, the model got 151 True Positives. It missed 121 Intermediate villi, predicting them as background, and confused 19 of them with Terminal villi. This suggests the model has trouble separating Intermediate villi when they overlap or appear in crowded areas.

It comes as no surprise that **Stem villi** were the hardest to detect. The model only correctly identified 16, while 19 were missed completely and labeled as background. This matches the lower precision and recall we saw earlier for this class and also can be due to their size and how their division during the tiling can lead to missing them in the detection or confusing them with other elements in the biological environment.

As for the **background class** (which ideally should never be predicted as villi), the model showed struggles as well. It predicted 119 background areas as Terminal, 73 as Intermediate, and 22 as Stem villi. This means the model sometimes sees villi where there aren't any or mistakes them, especially in dense or noisy regions.

5.3.4.2 F1 vs Recall

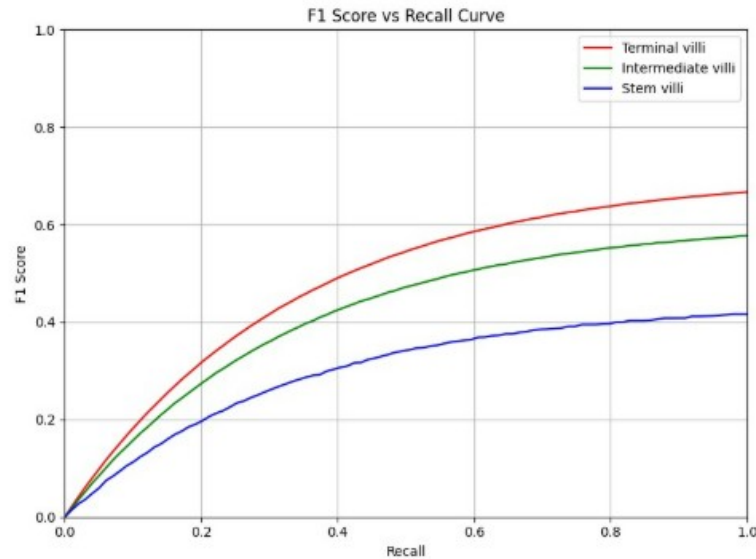


Figure 5.9: Confusion matrix of Faster R-CNN

The F1 vs Recall curve shows how well the model balances precision and recall across different thresholds. For Terminal villi, the curve is strong and smooth. The F1 score keeps improving and peaks around 0.66, which shows the model is quite good at detecting this villus type reliably with the current settings. Intermediate villi identification is not as strong as Terminal, but still consistent and promising, meaning the model can still detect most of them well, though it might get confused sometimes, especially around unclear boundaries or overlaps. Stem villi have demonstrated the weakest curve as usual. It's flat and shows a bit of noise, which means the model struggles. This suggests that it finds it hard to detect Stem villi confidently, most likely because they're less common(class imbalance), or look morphologically similar to adjacent villi.

5.3.4.3 Conclusion

The Faster R-CNN model achieved solid performance overall, particularly in detecting Terminal and Intermediate villi with reasonably high precision and stable F1 scores. However, it struggled with Stem villi due to class imbalance and morphological ambiguity, as reflected in its lower mAP and noisier recall behavior. These results highlight Faster R-CNN’s strength in precision and class separation, but also its conservative nature in underdetecting rarer classes.

5.4 Performance Comparison between YOLOv8s and Faster-Rcnn

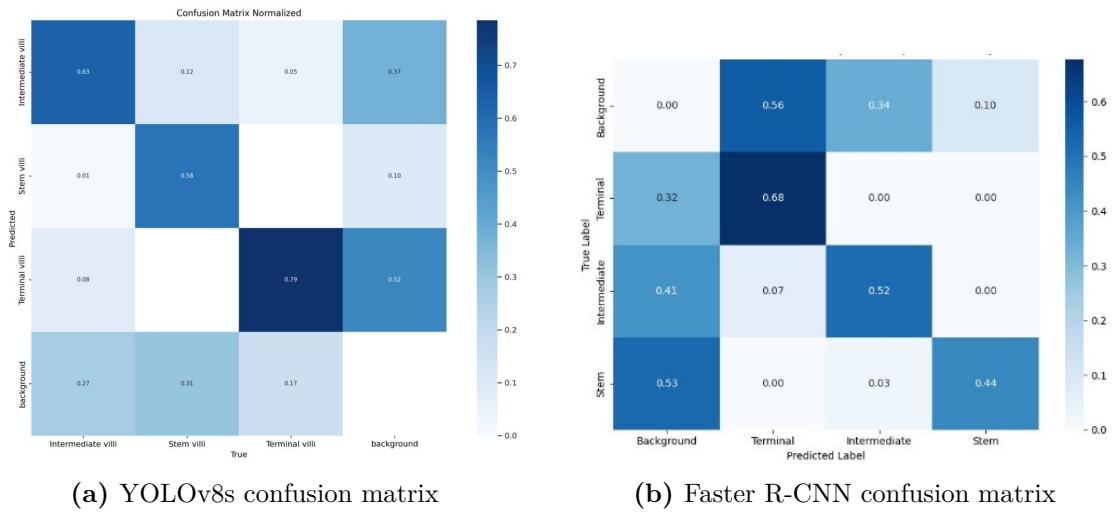


Figure 5.10: Comparison of confusion matrices between YOLOv8s and Faster R-CNN on the test set.

Analysis : Comparing the normalized confusion matrices reveals clear differences in how the two models behave. **Faster R-CNN**, shown in the second matrix, takes a more careful and selective approach. It correctly classifies 68% of Terminal villi and 58% of Stem villi, with very few mistakes between classes. Notably, it makes no confusion between Terminal and Intermediate, which suggests strong class separation. However, this precision comes with a downside: it misses 41% of Intermediate and 53% of Stem villi, misclassifying them as background. This shows that Faster R-CNN tends to underdetect harder or less frequent cases.

In contrast, **YOLOv8s** is more recall-oriented. It detects 63% of Intermediate villi, but also confuses some of them, 12% are mistaken for Stem, and 37% end up labeled as background. It performs even better on Terminal villi, correctly predicting 79% , but this comes with a high number of false positives—52% from the background are wrongly identified as Terminal.

This illustrates a classic trade-off: **YOLOv8s** is more aggressive as it prioritizes higher recall, catching more villi but also introduces more inaccuracies. **Faster R-CNN**, on the other hand, favors cleaner predictions, with fewer false positive rate but more missed detections. The choice between them depends on the use case. Faster R-CNN may be better where misclassifying structures carries a higher risk,

while YOLOv8s suits scenarios where catching as many relevant instances as possible is the priority.

Insight : Although the two models YOLOv8s and Faster-RCNN are built very differently and use different strategies for detection, both gave promising results, even under tough conditions like class imbalance, densely packed objects, and noisy biological textures.

YOLOv8s is a fast, single-stage model that doesn't use anchor boxes. Its main focus and strength are speed and detecting as many objects as possible. Thanks to its lightweight design, it's great for real-time inference and has managed to detect Terminal villi and smaller structures very well on 1024×1024 tiles. However, this speed came with its trade-offs; the model often confused background with actual villi and struggled more with distinguishing Intermediate villi. This aligns with the fact that YOLO skips region refinement used in two-stage detectors, and instead relies on fixed-grid predictions and confidence thresholding.

On the other hand, Faster R-CNN by employing a two-stage detection strategy, first identifies potential regions of interest before classifying them, which allows to have more precision on where and what to detect. Its Region Proposal Network (RPN) and feature pyramid backbone help the model understand spatial structure better, which comes in handy when we are implementing larger 2048×2048 tiles, which include more context. This careful approach helped the model reduce false positives and produce accurate predictions when detecting intermediate villi. However, it missed more Stem villi due to their scarcity and irregular structure, which was reflected in the model's cautious prediction tendencies.

The confusion matrices and precision-recall curves reflects these contrasts clearly. YOLOv8s detects more villi overall but is more prone to higher classification errors, while Faster R-CNN adapts a conservative approach and avoids false positives. The uneven class distribution, the low number of Stem villi, made these patterns more noticeable and highlighted how model architecture and training interact with data limitations.

To sum up, both models work well for this medical detection task, but in different ways. YOLOv8s is the most suitable when you want to detect more objects in a timely manner, while Faster R-CNN is better when your main focus is on being accurate and avoiding errors. Choosing between them really depends on what matters more for the use case, catching everything or minimizing errors. For future work, it would be useful to explore hybrid methods, balanced training, or models that adapt better to complex tissue patterns and rare classes like Stem villi.

Chapter 6

Conclusion

This thesis focused on building and evaluating two different object detection pipelines for identifying placental villi in histological images using YOLOv8 and Faster R-CNN. To support this, a high quality dataset was prepared by combining polygon-based XML annotations with mapped villi types, managing large slide dimensions through tiling strategies, and tackling class imbalance through oversampling and data augmentation. We then trained both a lightweight detector and a region-based one to identify the different types of villi.

Through our experiments, we found that each model brings something different to the table. YOLOv8s was faster and worked well on smaller, tightly packed structures like Terminal villi. On the other hand, Faster R-CNN proved to be more precise on larger, more complex tissue areas. These results show that both models have value depending on the specific features of the task, and they highlight how choosing or even combining models based on the problem can make a big difference. In real world use, the choice between the two models should be guided by the operational goals and priorities of the detection task, whether the motivation is maximizing throughput and sensitivity or ensuring precision and interpretability. Their complementary nature opens up the potential for hybrid approaches that combine their respective advantages more effectively.

Despite these encouraging results, we had faced several limitations. Class imbalance, differences in tile resolutions, and the intricate structure of placental tissue posed challenges to a good and consistent detection performance. These factors led to reduced sensitivity, particularly for scarce villi types.

Future work will focus on resolution-aware model training adapting input sizes to the morphological characteristics of each villus type and on exploring hybrid or ensemble architectures that combine the strengths of YOLOv8's speed and Faster R-CNN's precision. Additional exploration of alternative object detectors tailored for biomedical imaging may further enhance accuracy and generalizability.

Within the wider scope of medical image analysis, this study adds to the growing field of applying object detection in medical imaging. Adapting fast, accessible models and testing them in a domain-specific context which in our case was the placental histology, we've shown that even general-purpose tools can be powerful when carefully tuned and we've laid a solid foundation for future research in this area.

Acknowledgements

I would like to extend my sincere thanks for the help and guidance that i had received from the doctors and pathologists at semmelweis university, especially Kulcsár Krisztina , for taking the time to provide highly precise annotations and Dr. Attila Fintha , whose guidance and passion inspired us and deepened our appreciation for histopathology. I also wish to thank my supervisors for their support and mentorship throughout this research. s I hope this marks my first stepping stone in the research world of Biotechnology.

Bibliography

- [1] U-net: Convolutional networks for biomedical image segmentation. *arXiv preprint arXiv:1505.04597*, 2015. URL <https://arxiv.org/abs/1505.04597>.
- [2] Faster r-cnn: Real-time object detection with region proposal networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 39(6):1137–1149, 2017. URL <https://arxiv.org/abs/1506.01497>.
- [3] Pre-trained cnns for malaria detection. *PeerJ*, 6, 2018. DOI: 10.7717/peerj.4568. URL <https://peerj.com/articles/4568/>.
- [4] Fcos: Fully convolutional one-stage object detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 9627–9636, 2019. URL https://openaccess.thecvf.com/content_ICCV_2019/html/Tian_FCOS_Fully_Convolutional_One-Stage_Object_Detection_ICCV_2019_paper.html.
- [5] Covid-19 detection with x-rays. *Computers in Biology and Medicine*, 122, 2020. DOI: 10.1016/j.compbiomed.2020.103792. URL <https://doi.org/10.1016/j.compbiomed.2020.103792>.
- [6] Artificial convolutional neural network in object detection and semantic segmentation. *Frontiers in Oncology*, 2021. DOI: 10.3389/fonc.2021.638182. URL <https://www.frontiersin.org/journals/oncology/articles/10.3389/fonc.2021.638182/full>.
- [7] Med-yolo: Lightweight medical detection framework. *IEEE Access*, 11, 2022. DOI: 10.1109/ACCESS.2022.9743506. URL <https://ieeexplore.ieee.org/document/9743506>.
- [8] Medical image analysis using deep learning algorithms. *Frontiers in Public Health*, 2023. DOI: 10.3389/fpubh.2023.1273253. URL <https://www.frontiersin.org/journals/publichealth/articles/10.3389/fpubh.2023.1273253/full>.
- [9] YOLO-MedFormer: A multi-task learning framework. *arXiv preprint arXiv:2312.07729*, 2023. URL <https://arxiv.org/abs/2312.07729>.
- [10] Real-time object detection in medical imaging using yolo models for kidney stone detection. *EAJCSIT*, 12(7), 2024. URL <https://eajournals.org/ejcsit/vol12-issue-7-2024/real-time-object-detection-in-medical-imaging-using-yolo-models-for-kidney-st>
- [11] Fine-tuning strategies for transfer learning in medical imaging. *arXiv preprint arXiv:2406.10050v1*, 2024. URL <https://arxiv.org/html/2406.10050v1>.

- [12] Lesnet: Skin lesion classification via augmentation & transfer learning. *Procedia Computer Science*, 245, 2024. DOI: 10.1016/j.procs.2024.04.014. URL <https://doi.org/10.1016/j.procs.2024.04.014>.
- [13] Machine learning for medical imaging. PubMed, n.d.. URL <https://pubmed.ncbi.nlm.nih.gov/28212054/>.
- [14] Medyolo: A medical image object detection framework. AI Models, n.d.. URL <https://www.aimodels.fyi/papers/arxiv/medyolo-medical-image-object-detection-framework>.
- [15] Brain tumor detection using yolov5 and faster r-cnn. *International Journal of Intelligent Systems and Applications in Engineering (IJISAE)*, n.d.. URL <https://www.ijisae.org/index.php/IJISAE/article/view/3796>.
- [16] Using yolov11 for tumor detection in medical imaging. Ultralytics Blog, n.d.. URL <https://www.ultralytics.com/blog/using-yolo11-for-tumor-detection-in-medical-imaging>.
- [17] A review of cnns for medical image analysis. *Journal of Neonatal Surgery*, n.d.. URL <https://www.jneonatal surg.com/index.php/jns/article/view/5117>.
- [18] Deep convolutional backbone comparison for automated pet quality. PMC, n.d.. URL <https://pmc.ncbi.nlm.nih.gov/articles/PMC7616552/>.
- [19] Training strategies for radiology deep learning models. PMC, n.d.. URL <https://pmc.ncbi.nlm.nih.gov/articles/PMC8637222/>.
- [20] Deep learning-based object detection in medical imaging. SSRN, n.d.. URL https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4866522.
- [21] Ai in medical imaging: Threat or opportunity? PMC, n.d.. URL <https://pmc.ncbi.nlm.nih.gov/articles/PMC7340949/>.
- [22] Yolov8: Ultralytics open-source framework. GitHub, n.d.. URL <https://github.com/ultralytics/ultralytics>.
- [23] Cell nuclei detection on whole slide histopathology images using histomicstk and faster r-cnn. Kitware Blog, n.d.. URL <https://www.kitware.com/cell-nuclei-detection-on-whole-slide-histopathology-images-using-histomicstk->
- [24] Yolov8 vs faster r-cnn: A comparative analysis. Keylabs Blog, n.d..
- [25] How artificial intelligence is shaping medical imaging technology. PMC, n.d.. URL <https://pmc.ncbi.nlm.nih.gov/articles/PMC10740686/>.
- [26] Machine learning in medical imaging. LinkedIn, n.d.. URL <https://www.linkedin.com/pulse/machine-learning-medical-imaging-umer-haddii-4mtnf>.
- [27] Faster r-cnn explained. Viso.ai, n.d.. URL <https://viso.ai/deep-learning/faster-r-cnn-2/>.
- [28] Faster r-cnn: How it works. ThinkAutonomous.ai, n.d.. URL <https://www.thinkautonomous.ai/blog/faster-rcnn/>.

- [29] Yolov8 medical imaging github repository. GitHub, n.d.. URL <https://github.com/sevdaimany/YOLOv8-Medical-Imaging>.
- [30] Satya Mallick. Ultralytics yolov8 - official guide. <https://learnopencv.com/ultralytics-yolov8/>, 2023. Accessed: 2025-05-22.
- [31] Zhaohui Zheng, Ping Wang, Wei Liu, Jinze Li, Rongguang Ye, Dongwei Ren, and Shiming Xiang. Distance-iou loss: Faster and better learning for bounding box regression. <https://arxiv.org/abs/1911.08287>, 2019. arXiv preprint arXiv:1911.08287.